



الجمهورية العربية السورية  
المعهد العالي للعلوم التطبيقية والتكنولوجيا  
قسم الاتصالات  
العام الدراسي 2010/2011

مشروع السنة الرابعة

## بناء نظام ذكي لإدارة الطلبات في المؤسسات الجامعية

تقديم

ليلى العبدالله

إشراف

د. مصطفى دقاق

ما. عماد قرحيلي

8/8/202

## الخلاصة

في ظل التطور السريع في مجال هندسة البرمجيات وأنظمة المعلومات، أصبحت الأنظمة الورقية والتقليدية القائمة على التوجيه اليدوي عائقاً أمام كفاءة العمليات الإدارية في العديد من المؤسسات والجامعات.

يعالج هذا المشروع مشكلة العملية الحالية المتبعة في تقديم الطلبات الإدارية ومعالجتها داخل المؤسسات الجامعية، والتي تعتمد بشكل كبير على المعرفة الشخصية بمسارات التوجيه، وتفتقر إلى آلية موحدة لتتبع حالة الطلب أو تحديد المسؤول عنه في كل مرحلة من مراحله، مما يؤدي إلى تأخر في الإنجاز، وضياح للطلبات، وصعوبة في المساءلة والتدقيق.

الهدف الأساسي من المشروع هو تصميم وتطوير نظام ويب ذكي، ثنائي اللغة (عربي/إنجليزي)، يهدف إلى أتمتة دورة حياة الطلبات الإدارية بالكامل: بدءاً من تصنيف الطلب تلقائياً اعتماداً على نموذج ذكاء اصطناعي مدرب على تصنيف النصوص العربية، مروراً بتوجيهه عبر مسار موافقات قابل للتهيئة الكاملة من قبل الإدارة، وصولاً إلى تقدير الوقت المتبقي لإنجازه بالاستناد إلى التعلم من سجل الطلبات التاريخي، مع توفير آلية تدخل بشري (Human-in-the-Loop) عند انخفاض ثقة النموذج في تصنيفه.

يعرض هذا العمل خطوات عملية بناء هذا التطبيق، ابتداءً من مراحل فهم منطق العمل وبناء لغة مشتركة ( Ubiquitous Language) بين فريق التطوير وأصحاب المصلحة، مروراً بمراحل التحليل والتصميم البرمجي، وصولاً إلى بناء تطبيق قابل للتوسع والاختبار والصيانة، قائم على مفهوم التصميم الموجه بالجال (Domain-Driven Design)، ومستخدماً البنية المعمارية النظيفة (Clean Architecture)، مع دمج خدمة ذكاء اصطناعي منفصلة للتصنيف النصي والتنبؤ الزمني.

# Abstract

In light of the rapid advancement in software engineering and information systems, legacy paper-based systems relying on manual routing have become an obstacle to the efficiency of administrative operations in many institutions and universities.

This project addresses the problems of the current process used for submitting and handling administrative requests within the university institution, which relies heavily on personal knowledge of routing paths and lacks a unified mechanism for tracking a request's status or identifying the responsible party at each stage, leading to delays in completion, lost requests, and difficulty in accountability and auditing.

The primary goal of the project is to design and develop an intelligent, bilingual (Arabic/English) web-based system aimed at fully automating the lifecycle of administrative requests: from automatically classifying each request using an AI model trained on Arabic text classification, through routing it along an approval workflow that is fully configurable by the administration, to estimating its remaining completion time based on learning from the historical request log, while providing a Human-in-the-Loop fallback mechanism when the model's classification confidence is low.

This work outlines the practical steps for building this application, starting from understanding the business logic and establishing a Ubiquitous Language between the development team and stakeholders, through the stages of software analysis and design, culminating in the construction of a scalable, testable, and maintainable application based on the concept of Domain-Driven Design, utilizing Clean Architecture, and integrating a separate AI microservice for text classification and remaining-time prediction.

## الفصل الأول

# التعريف بالمشروع

يتضمن هذا الفصل التعريف بالمشروع ومتطلباته.

### 1.1- مقدمة

تعدّ عملية معالجة الطلبات الإدارية داخل المؤسسات الجامعية من العمليات الجوهرية التي تمس تجربة جميع الأطراف المعنية طلاباً وموظفين وإدارة، إذ تمثل نقطة التقاء يومية بين مقدّم الطلب والجهات المسؤولة عن دراسته والموافقة عليه، اعتماداً على مجموعة من القواعد والمسارات التنظيمية الداخلية. في ظل التطور السريع في مجال هندسة البرمجيات وظهور تقنيات حديثة في معالجة اللغة الطبيعية والتعلم الآلي توفر أداءً عالياً وتجربة مستخدم أفضل، أصبحت الطريقة التقليدية اليدوية المتبعة حالياً في توجيه الطلبات ومتابعتها قديمة ومحدودة القدرات، إذ تعتمد بشكل كبير على المعرفة الشخصية بمسارات التوجيه، ولا توفر أي آلية موحدة لتتبع حالة الطلب أو تقدير الوقت اللازم لإنجازه. من هذا المنطلق، جاء المشروع كحل بديل لأتمتة دورة حياة الطلب الإداري بالكامل، بدءاً من تقديمه وحتى إنجازه. يهدف المشروع إلى تقديم نظام ويب ذكي وثنائي اللغة (عربي/إنجليزي)، وذلك بهدف أتمتة تصنيف الطلبات وتوجيهها عبر مسار الموافقات المناسب، وتقدير الوقت المتبقي لإنجازها، وتقديم واجهات سهلة الاستخدام تعزز من كفاءة العمل وشفافيته.

### 2.1- الهدف من المشروع

إن الهدف الأساسي من المشروع هو إنشاء نظام برمجي متكامل وموثوق لأتمتة دورة حياة الطلبات الإدارية داخل المؤسسة، مما يؤدي إلى تحسين الكفاءة والدقة وتقليل الاعتماد على التدخل اليدوي في التوجيه والتصنيف. يسعى النظام لتحقيق ذلك من خلال مركزية معالجة الطلبات عبر توفير تطبيق موحد يجمع كافة الوظائف (تقديم الطلبات، التصنيف الآلي، التوجيه عبر المسارات المحددة، متابعة الحالة، توليد التقارير) في منصة واحدة سهلة الوصول. كما يركز النظام على أتمتة استقبال الطلبات الواردة وتصنيفها فور تقديمها اعتماداً على نموذج ذكاء اصطناعي مدرب لتصنيف النصوص العربية، حيث يتم تتبع حالة كل طلب بشكل دقيق ولحظي، وتوجيهه تلقائياً إلى المسؤول المناسب في كل خطوة بحسب قواعد التوجيه المهيأة والمهل الزمنية المحددة لكل خطوة (SLA)، لضمان معالجة الطلبات ضمن الترتيب والأولوية الصحيحين. مع توفير آلية تدخل بشري

(Human-in-the-Loop) عند انخفاض ثقة النموذج، مع ضمان التطبيق الدقيق لقواعد الأهلية والموافقة الداخلية المهيأة من قبل الإدارة. ولدعم اتخاذ القرار، سيوفر النظام القدرة على إنشاء تقارير تحليلية إدارية توضّح أداء سير العمل ومدد الإنجاز والطلبات المتأخرة، وأخيراً، سيحقّق التكامل الآمن مع أنظمة المؤسسة الأخرى (كنظام شؤون الموظفين) عبر واجهات برمجية آمنة (APIs) لمزامنة البنية التنظيمية.

### 3.1- المتطلبات الوظيفية

1. يجب أن يسمح النظام للمسؤول عن النظام بـ:

1.1. إدارة حسابات المستخدمين.

1.2. إدارة الهيكل الإداري للمؤسسة.

1.3. إدارة صلاحيات المستخدمين.

1.4. إدارة قوالب الطلبات المختلفة.

1.5. إدارة مسارات الطلبات.

2. يجب أن يسمح النظام للمستخدم بـ:

2.1. إنشاء طلب جديد وإرفاق ملفات معه إن وُجد.

2.2. إرسال طلب بحسب صلاحياته للجهة المناسبة وبحسب المسارات المحددة مسبقاً.

2.3. تتبع مسار الطلب الخاص به.

2.4. قبول الطلب أو رفضه أو إضافة تعليق عليه، بحسب صلاحياته، وإرفاق ملفات معه إن وُجد.

3. يجب أن يسمح النظام لمدير النظام بـ:

1.1. طلب إصدار تقارير وإحصائيات (سنوية، شهرية، أسبوعية...) حول أداء النظام مثل حجم

المراسلات، استجابة المستخدمين، سرعة الإنجاز ومقارنة الإنجاز بين الأقسام.

- 1.2. البحث عن طلب معين بحسب رقمه، والقدرة على استعراض مسار الطلب.
4. يجب أن يقوم النظام بإرسال إشعارات فورية للمستخدمين عند قبول طلباتهم أو رفضها، وعند وصول طلبات إليهم.
5. يجب أن يكون النظام قادر على فهم مضمون الطلب وتحويله للمسار المناسب، وفي حال عدم القدرة على فهم مضمونه يتم تحويله لموظف مختص يقوم بإسناده لمسار مناسب يدوياً.
6. يجب أن يسمح النظام للمستخدمين باستعراض الطلبات الخاصة بهم.

## 4.1- المتطلبات غير الوظيفية

1. الأداء والاستجابة (Performance & Responsiveness)
- زمن التنقل بين واجهات النظام: يجب أن يتم تحميل الواجهات للتفاعل مع المستخدم (مثل فتح طلب أو تنقل في لوحة التحكم) في أقصر زمن ممكن.
  - زمن معالجة الذكاء الاصطناعي: يجب أن تقوم عمليات التصنيف والتوجيه والتنبؤ بزمن انتهاء معالجة البيانات وإرجاع النتيجة في أقل زمن ممكن لتحسين تجربة المستخدم.
2. الأمان والخصوصية (Security & Privacy)
- المصادقة والتفويض: يجب أن يطبق النظام تحكماً في الوصول القائم على الأدوار (RBAC)، وتشفير كلمات المرور.
  - قابلية التدقيق: يجب تسجيل كل إجراء (إنشاء، تحويل، قبول، رفض طلب) مع طابع زمني، ومعرف المستخدم، وعنوان IP لإنشاء "سجلات الأحداث (Event Logs)" اللازمة للنموذج الذكي والتدقيق الإداري.
3. الموثوقية والتوافر (Reliability & Availability)
- التوافر: يجب أن يكون النظام متوافراً خلال ساعات العمل المؤسسية.

- التسامح مع الأخطاء (Fault Tolerance): إذا فشلت خدمة الذكاء الاصطناعي أو توقفت، يجب أن يتراجع النظام بمرونة ويقوم تلقائياً بتحويل الطلب إلى "الموظف المختص" (الإسناد اليدوي) بدون تأخير.

#### 4. سهولة الاستخدام (Usability)

- يجب أن تكون واجهات المستخدم واضحة، وسهلة الفهم والاستخدام.

#### 5. القابلية للتوسع والصيانة (Scalability & Maintainability)

- يجب أن يكون النظام مصمماً بطريقة مرنة تسمح بإجراء تحديثات وإضافة وظائف جديدة مستقبلاً بسهولة، بالإضافة إلى سهولة صيانتها في حال حدوث أي خلل.

#### 6. أداء نموذج الذكاء الاصطناعي (AI Models Performance)

- عتبة الثقة (AI Confidence Threshold): يجب أن يحتوي نظام التصنيف والتوجيه الذكي على عتبة ثقة قابلة للتحقيق (80%). إذا كانت ثقة النموذج في توجيه الطلب أقل من هذه العتبة، يجب تفعيل التوجيه اليدوي (Human-in-the-loop).

#### 7. المعايير التقنية (Technical Standards)

- يجب أن تكون قواعد البيانات مورفولوجية لضمان ديناميكية محددات الطلبات دون الحاجة إلى إعادة البرمجة.

#### 8. مؤشرات الأداء والتقييم (KPIs & Evaluation Metrics)

لقياس مدى نجاح النظام في سد الفجوة التقنية وتحقيق الأهداف المذكورة في الدراسة المرجعية، يتم الاعتماد على المؤشرات التالية:

##### 1- مؤشرات جودة النماذج الذكية (AI Model Performance)

- دقة التوجيه الآلي (NLP Accuracy): النسبة المئوية للطلبات التي يتم توجيهها بشكل صحيح آلياً (80% مثلاً).
- معدل المسار البديل (HITL Rate): نسبة الطلبات التي تتطلب تدخلاً بشرياً عند انخفاض عتبة الثقة (أقل من 20% مثلاً).

## 2- مؤشرات الكفاءة التشغيلية (Operational Efficiency)

- زمن الدوران (Turnaround Time): متوسط الوقت الكلي من لحظة إنشاء الطلب حتى إغلاقه، مقارنة بالأنظمة التقليدية.
- زمن استجابة التوجيه (Routing Latency): تقليص الوقت المستغرق لتحويل المعاملة من ساعات (في التوجيه البشري) إلى ثوانٍ معدودة (عبر التوجيه الذكي).

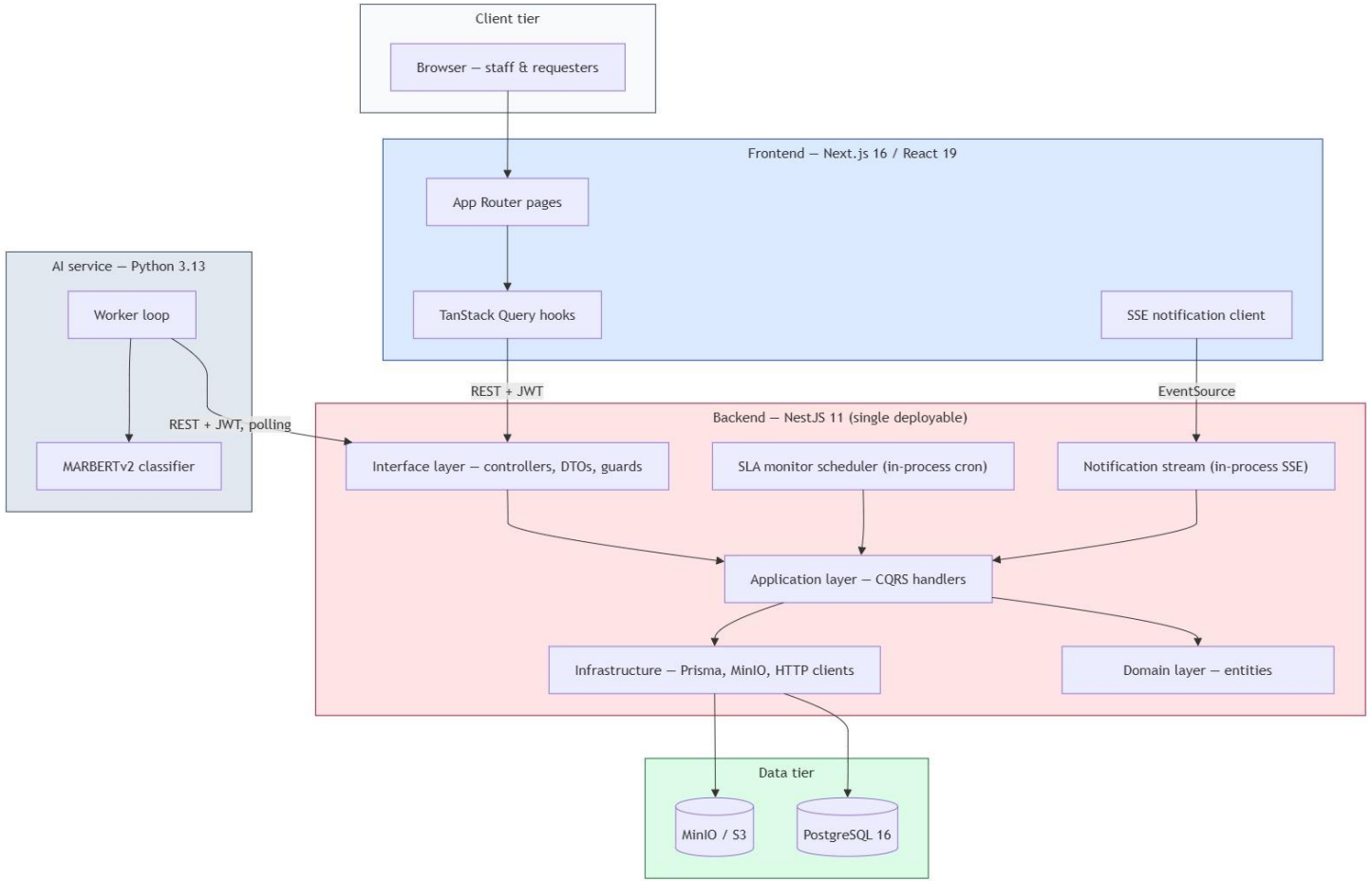
## الفصل الثاني

# المخططات

يتضمن هذا الفصل مخططات تصميم النظام.

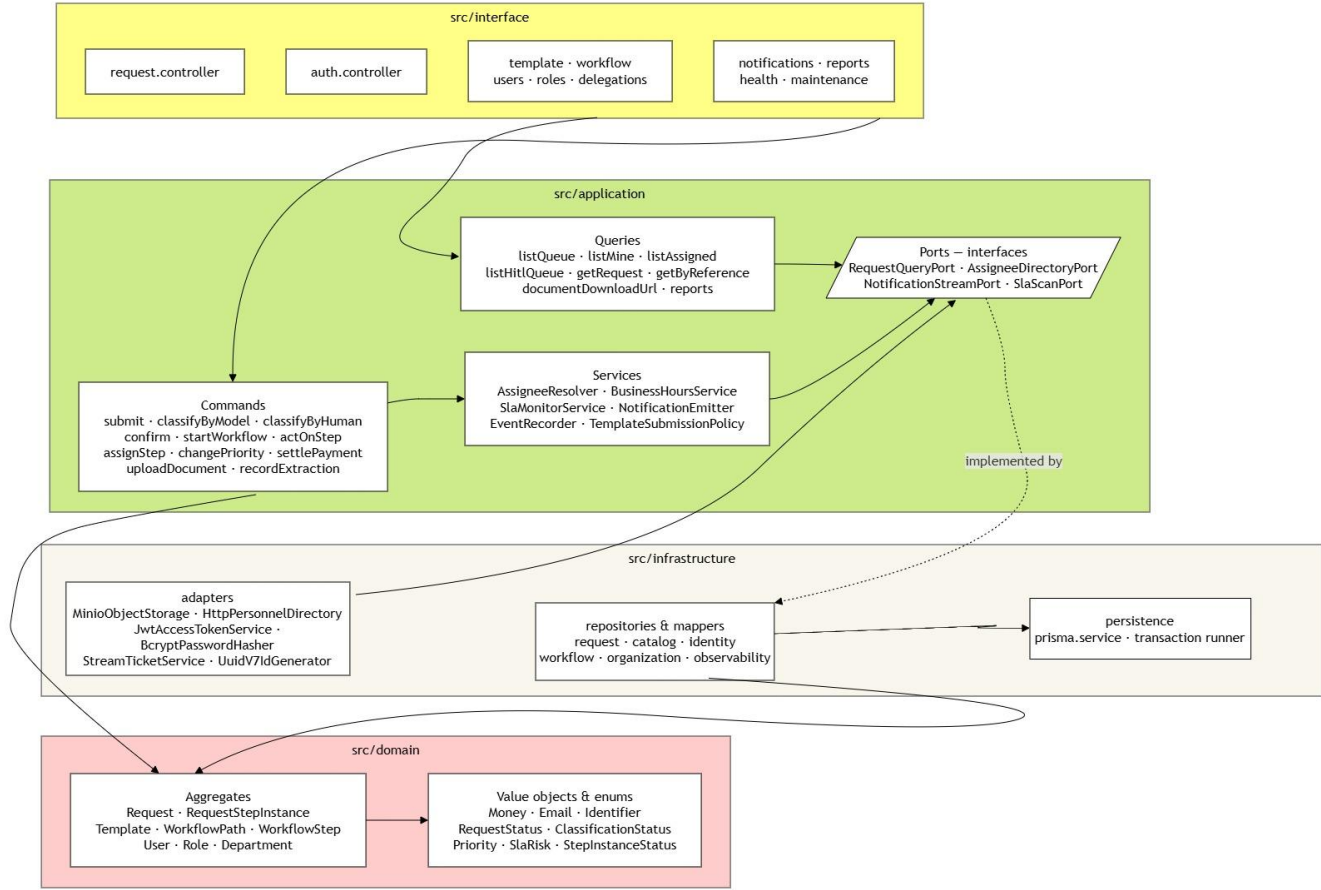
### System Block Diagram -1.2

يبيّن هذا المخطط الحدود الفيزيائية للنظام حيث يتكون النظام يتكوّن من خمسة أجزاء: ال Frontend (Next.js)، وال Backend API (NestJS)، وقاعدة PostgreSQL، وخدمة التخزين MinIO المتوافقة مع S3، وخدمة ال AI المكتوبة بلغة Python، يُلاحظ أن ال SLA monitor وقناة ال notifications ليستا خدمتين منفصلتين، بل تعملان in-process داخل ال Backend. هذا لأن كليهما تحتاج الوصول إلى ال repositories نفسها وإلى ال transaction runner نفسه، وفصلهما كان سيُنتج شبكة استدعاءات دون أي مكسب في الأداء أو في العزل.



## 2.2- مخطط المعمارية الطبقيّة للـ Backend (Layered / Ports Diagram)

يوضح هذا المخطط الطبقات الأربع واتجاه الاعتماد بينها: domain ← application ← interface، مع infrastructure التي تُحقق الـ ports ولا يعتمد عليها أحد، الأسهم كلها تتجه نحو الداخل.

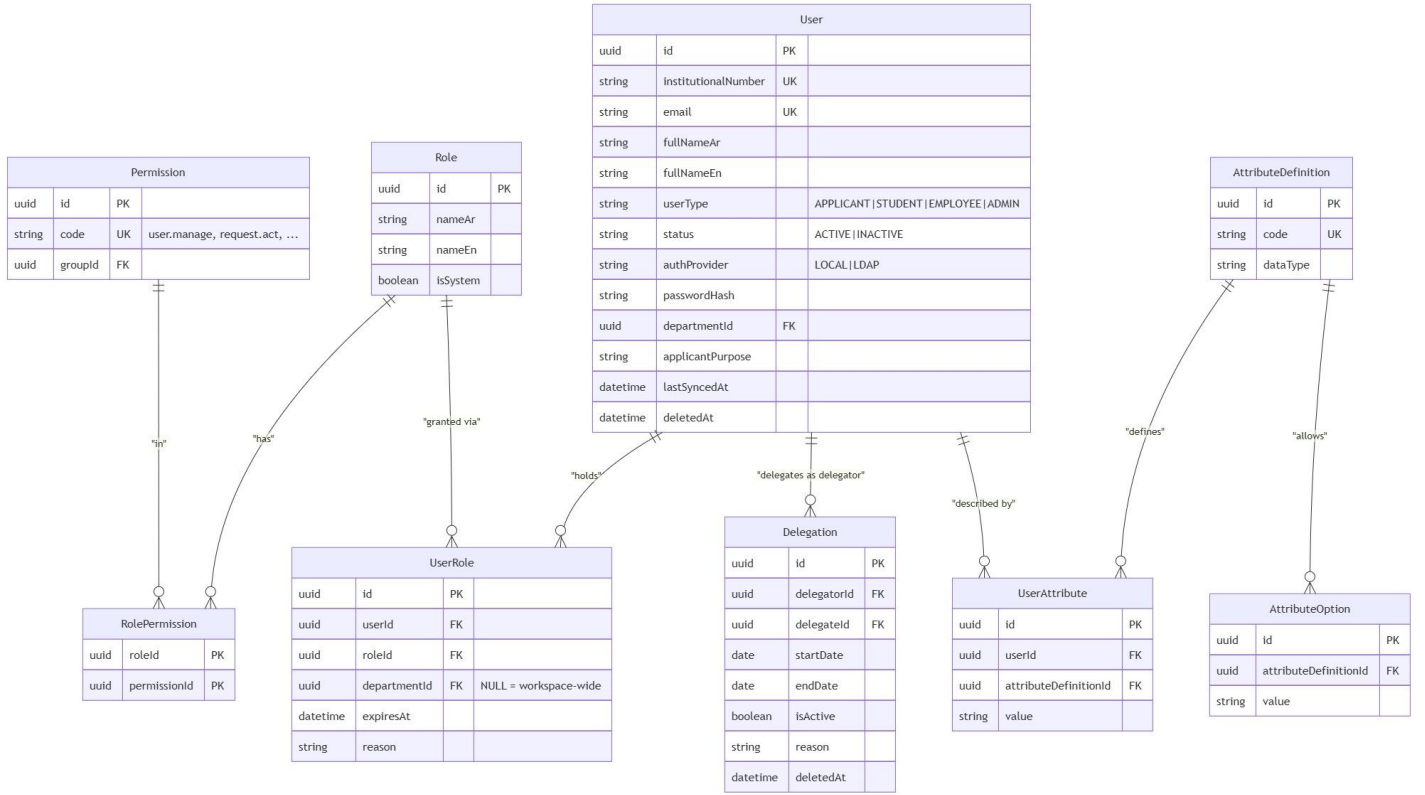


## 2.2- ERD (مخطط الكيانات والعلاقات)

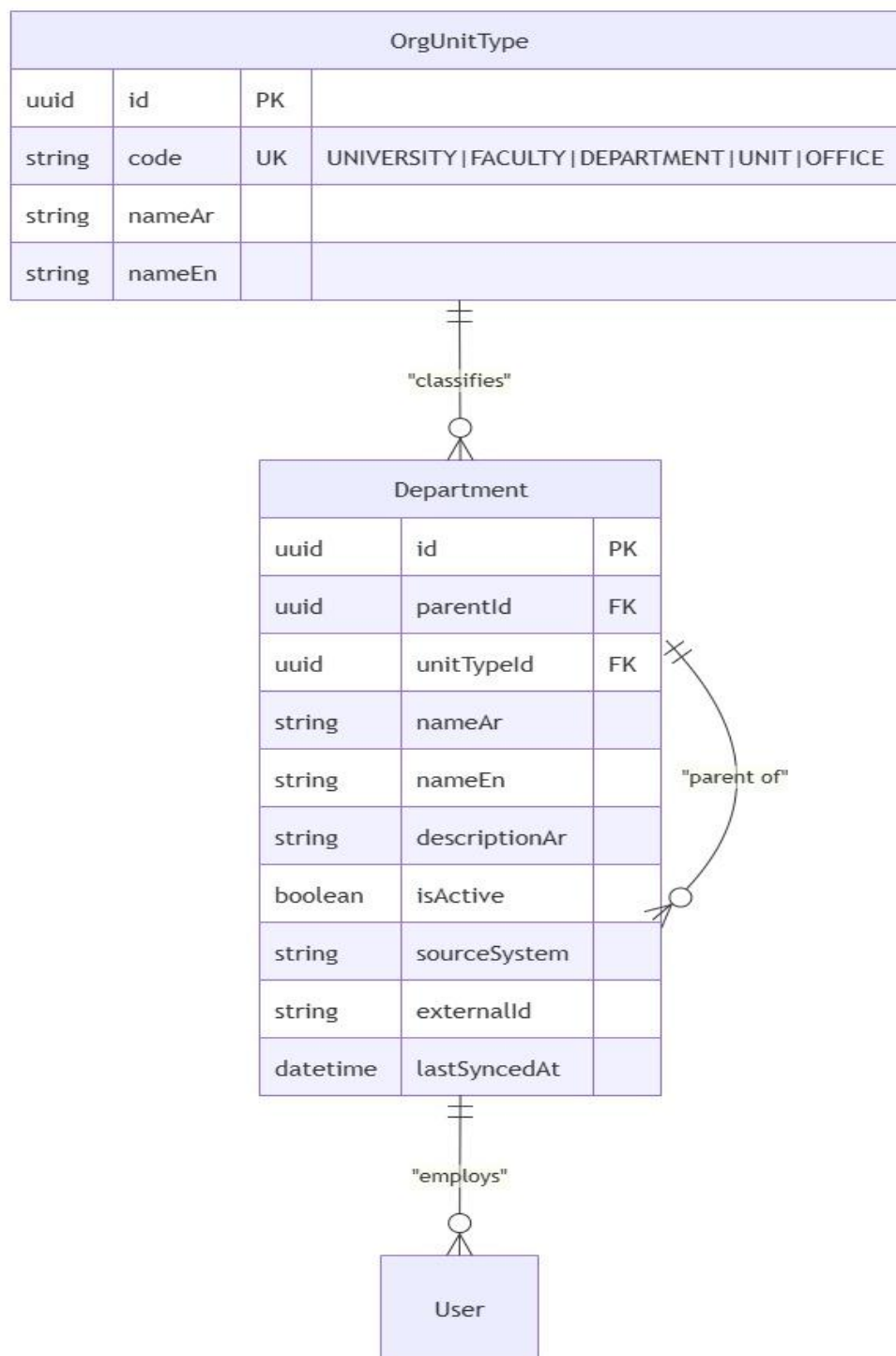
يمثل هذا المخطط البنية الفعلية لقاعدة البيانات، أي خمسة وثلاثين model موزعة على ثمانية سياقات: الهوية والصلاحيات، البنية التنظيمية، catalog، مسارات العمل، الطلبات، المراقبة والـ AI.

يوجد ثلاث نقاط أساسية في هذا المخطط. أولاً، جدول UserRole يحمل departmentId و expiresAt خاصين به، أي أن الصلاحية تُمنح مقيدة بوحدة تنظيمية ومحددة زمنياً، وهذا الحقل تحديداً هو أساس محرك التوجيه. ثانياً، جدول WorkflowStep يحمل feeAmount و feeCurrency، أي أن الرسم خاصية للخطوة لا للطلب. ثالثاً، الفصل بين WorkflowStep (التعريف) و RequestStepInstance (النسخة المنقذة) هو ما يسمح بتعديل مسار العمل (ال workflow) دون المساس بالطلبات الجارية.

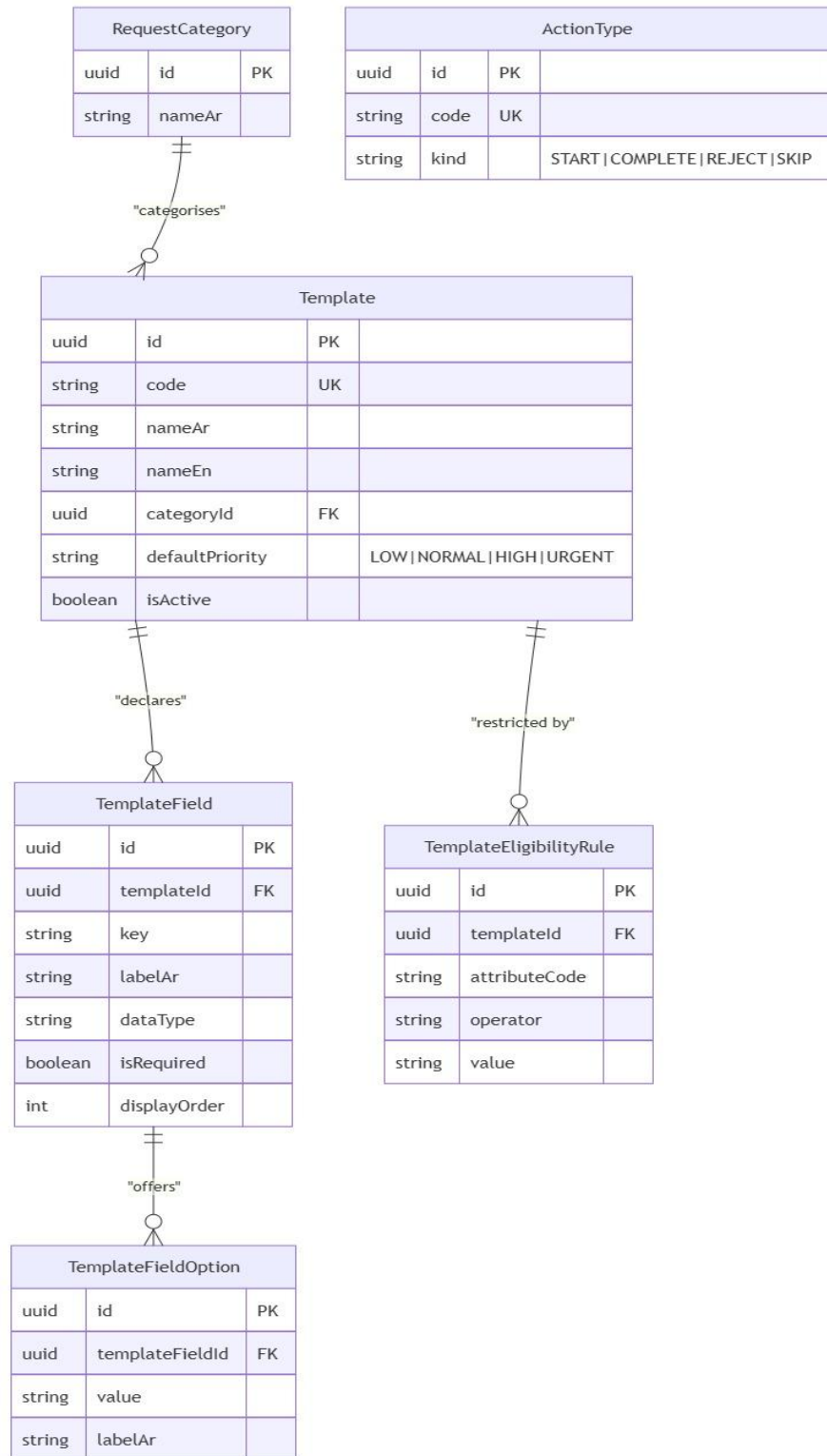
# 1. الهوية والصلاحيات:



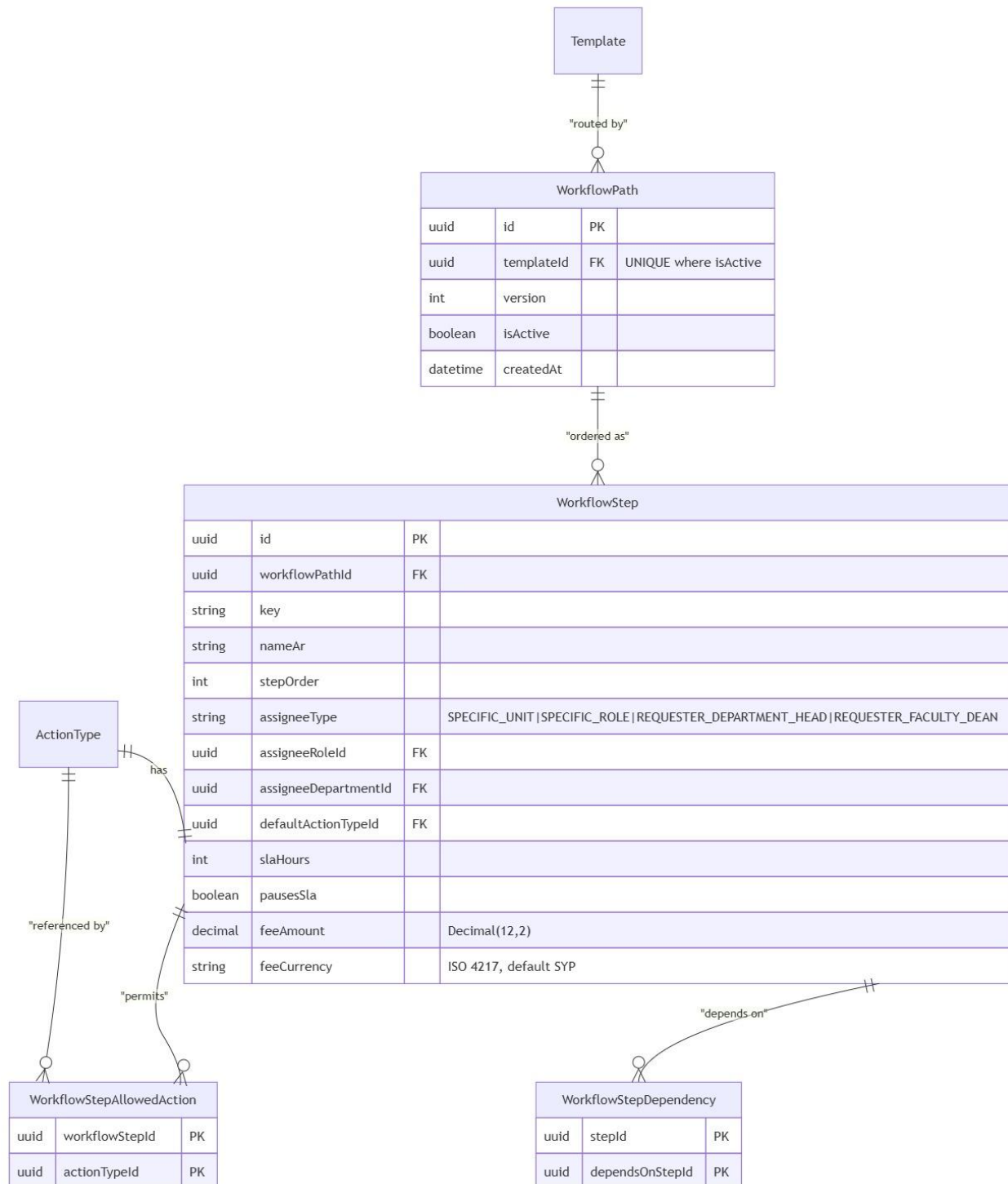
## 2. البنية التنظيمية:



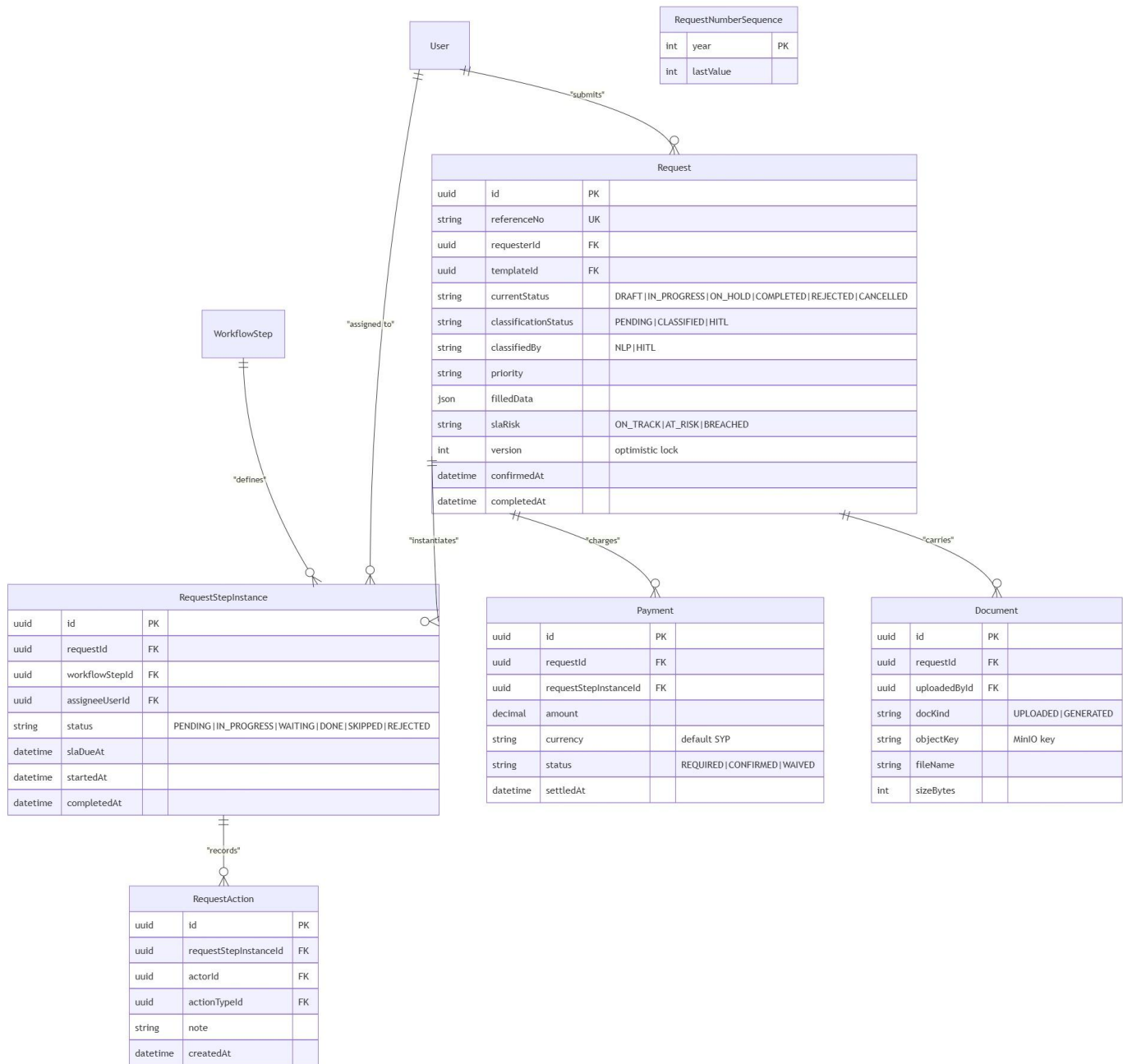
### : Catalog .3



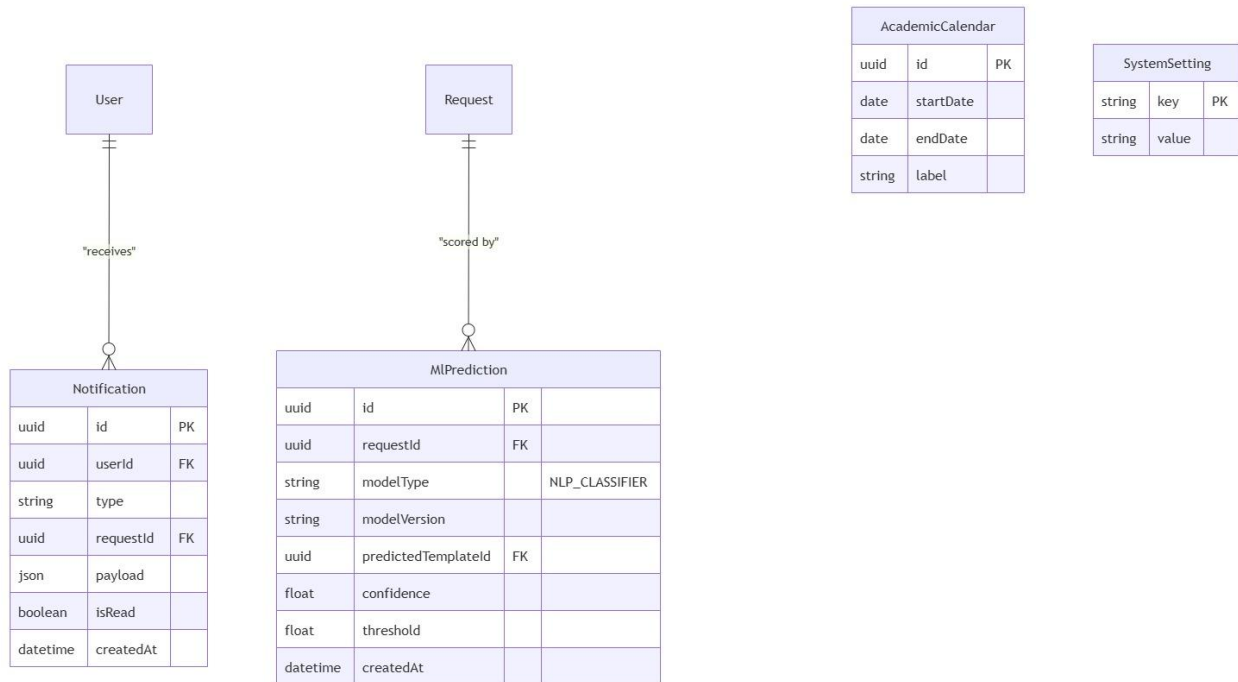
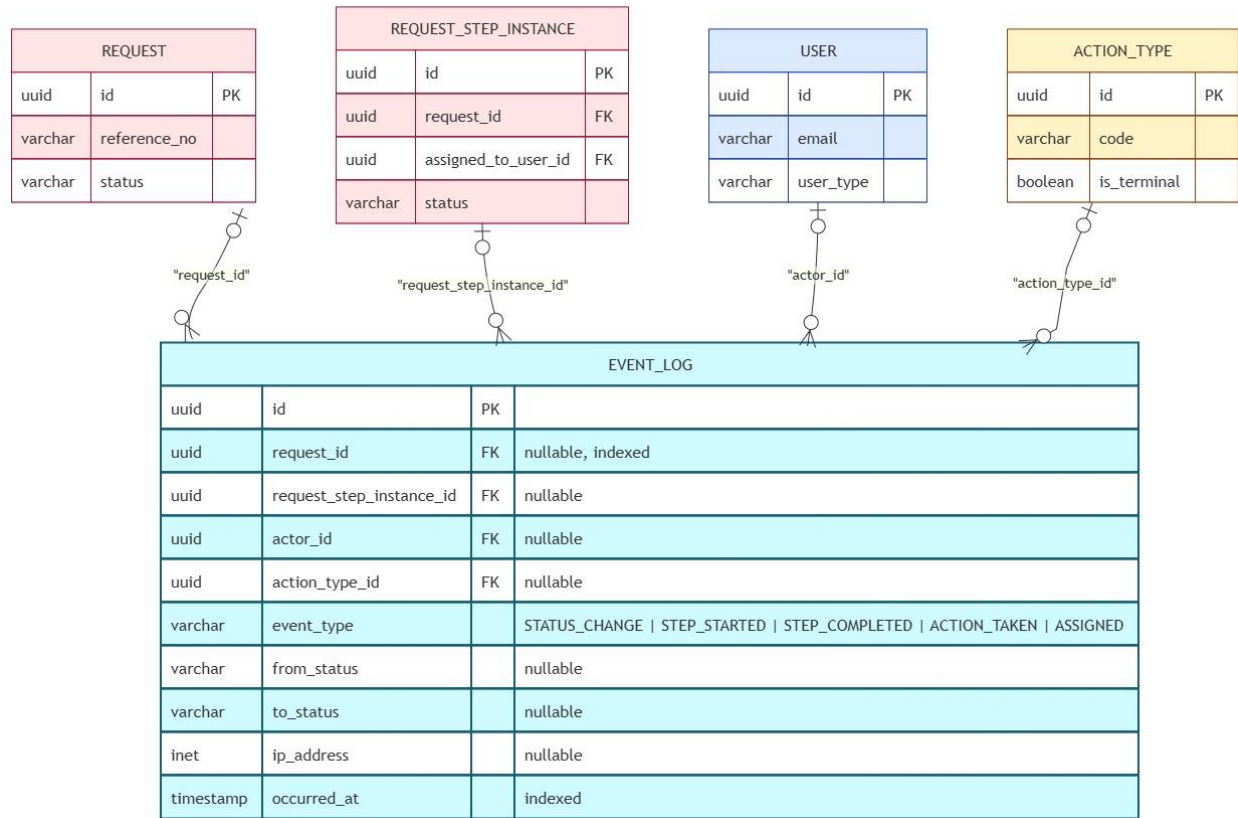
#### 4. مسار العمل:



## 5. الطلب:



## 6. المراقبة والـ AI:



## 3.2 – Class Diagram (مخطط الصفوف)

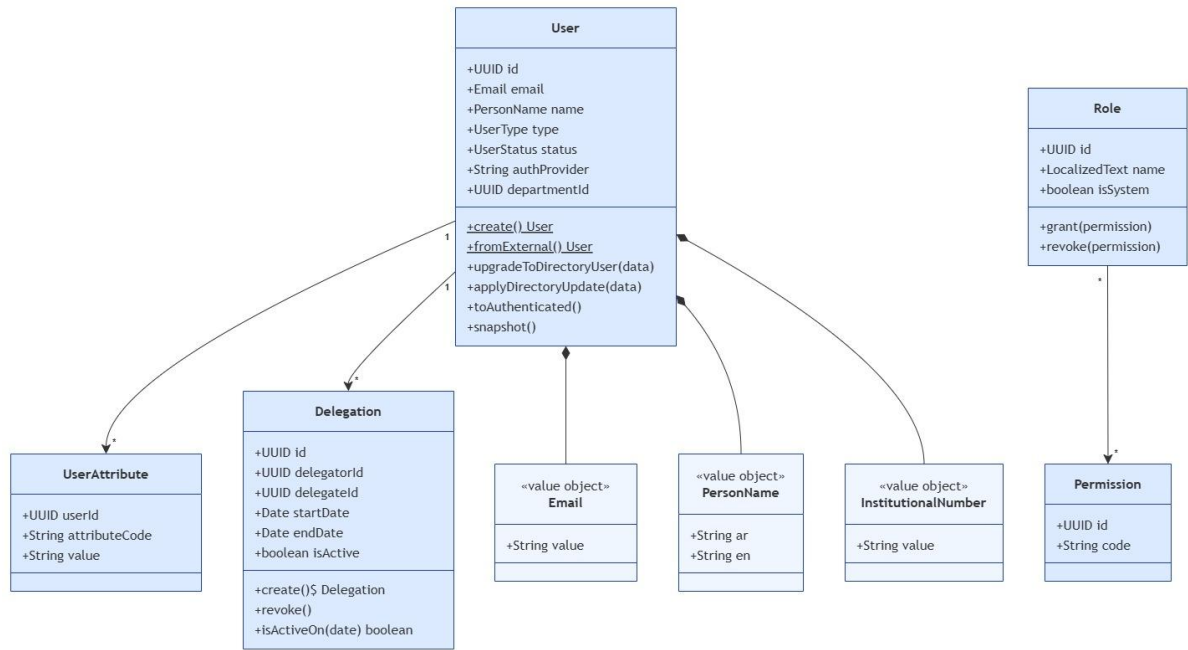
في هذا المشروع، تم تبني فصل صارم بين الـ ERD والـ Domain Class Diagram. فبينما يصف الـ ERD آلية التخزين (Data Persistence) (كيفية تخزين البيانات بشكل علائقي في PostgreSQL)، يصف الـ Domain Model السلوك والصلاحيات (Behavior and Authority) (من يملك قرار التعديل وتطبيق القواعد).

تعتمد المعمارية على مبادئ (Domain-Driven Design (DDD)، حيث تُستخدم الـ Aggregates (مثل Request و WorkflowPath و Template) كحدود للاتساق (Consistency Boundaries). يعمل الـ Aggregate Root كجذابة حصرية للبيانات الداخلية الخاصة به؛ فعلى سبيل المثال، لا يمكن تعديل خطوة (Step) في مسار العمل بشكل منعزل، بل يجب أن يمر التعديل حصراً عبر الـ Aggregate المسمى WorkflowPath لضمان صحة العملية.

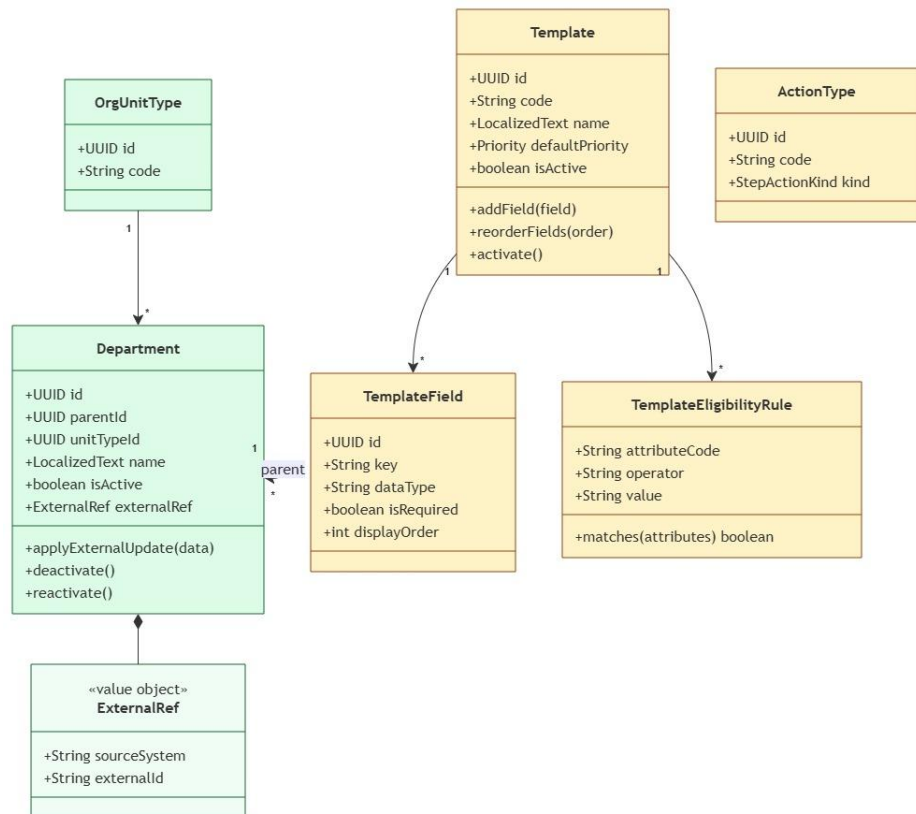
علاوة على ذلك، يطبق النظام مفهوم Rich Domain Model. حيث تم وضع القواعد المنطقية (Invariants) وقيود العمل التجاري داخل الـ Aggregates نفسها على شكل Methods واضحة (مثل Request.complete () و WorkflowPath.activate ())، بدلاً من تشتيتها داخل الـ Application Services. على سبيل المثال، عند استدعاء WorkflowPath.activate ()، يقوم الـ Aggregate داخلياً بتنفيذ خوارزمية Kahn (Kahn's Algorithm) لضمان عدم وجود مسارات حلقية (Acyclic Graph) قبل تغيير حالته إلى فعال.

من خلال تغليف هذه القواعد (Encapsulation) داخل كيانات المجال، تضمن المعمارية أن تمثيل الحالة غير الصحيحة (Invalid State) هو أمر مستحيل برمجياً. فالنظام لا يعتمد فقط على الـ Services "للتذكر" التحقق من الشروط، بل إن كائنات المجال (Domain Objects) نفسها ترفض فعلياً أي انتقال غير قانوني للحالة، مما يضمن سلامة البيانات (Data Integrity) المطلقة بغض النظر عن كيفية تعامل الـ Application Layer معها.

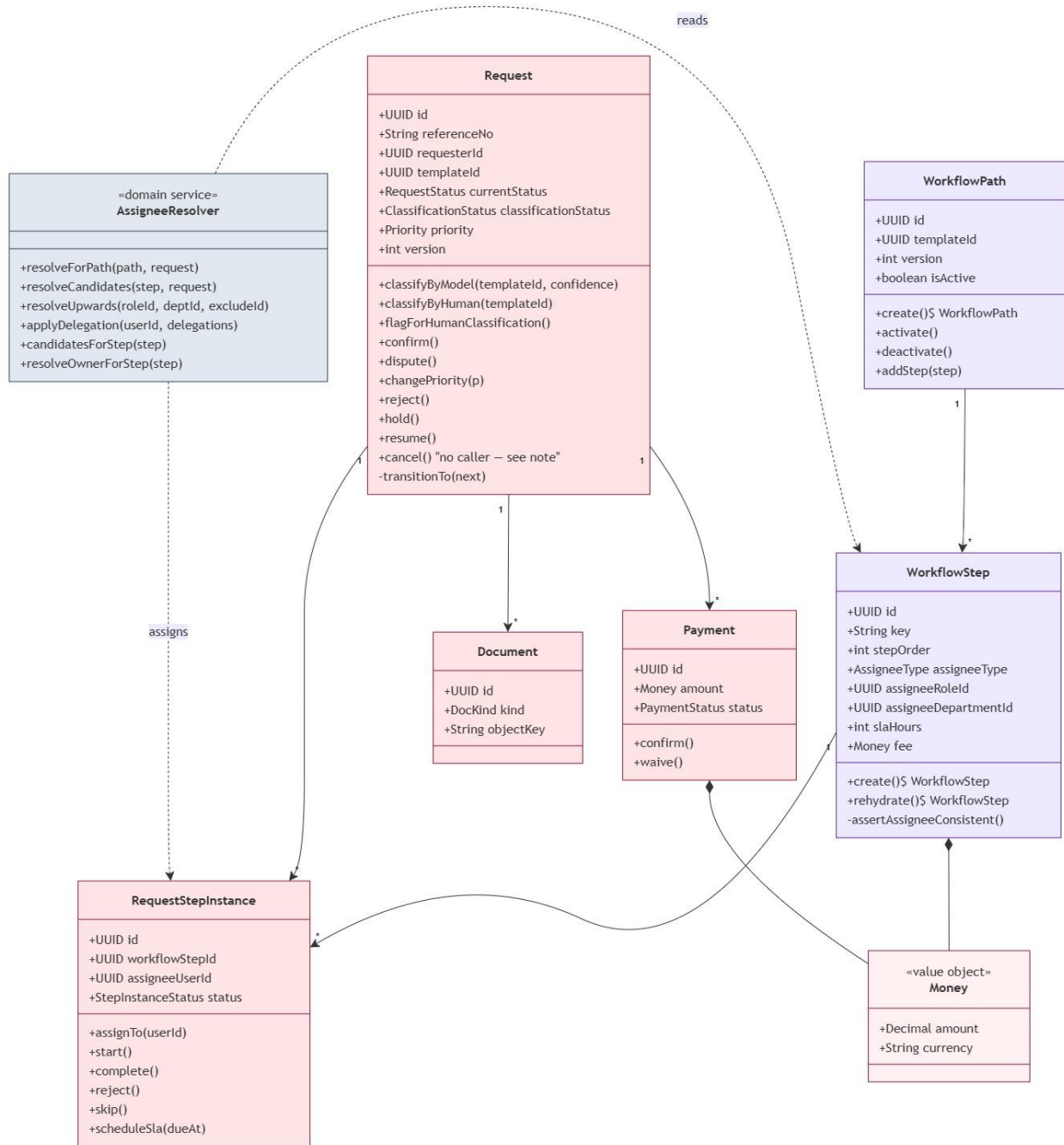
## 1. الهوية والصلاحيات:



## 2. البنية التنظيمية وال catalog:



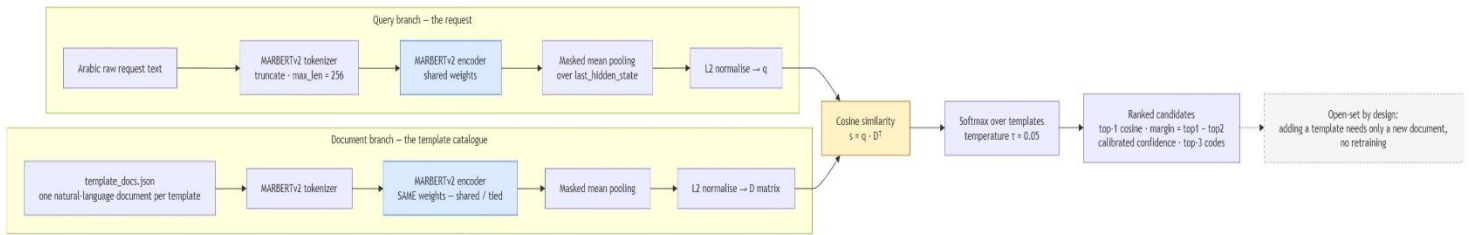
### 3. مسار العمل والطلبات:



## AI Model Diagram –4.2

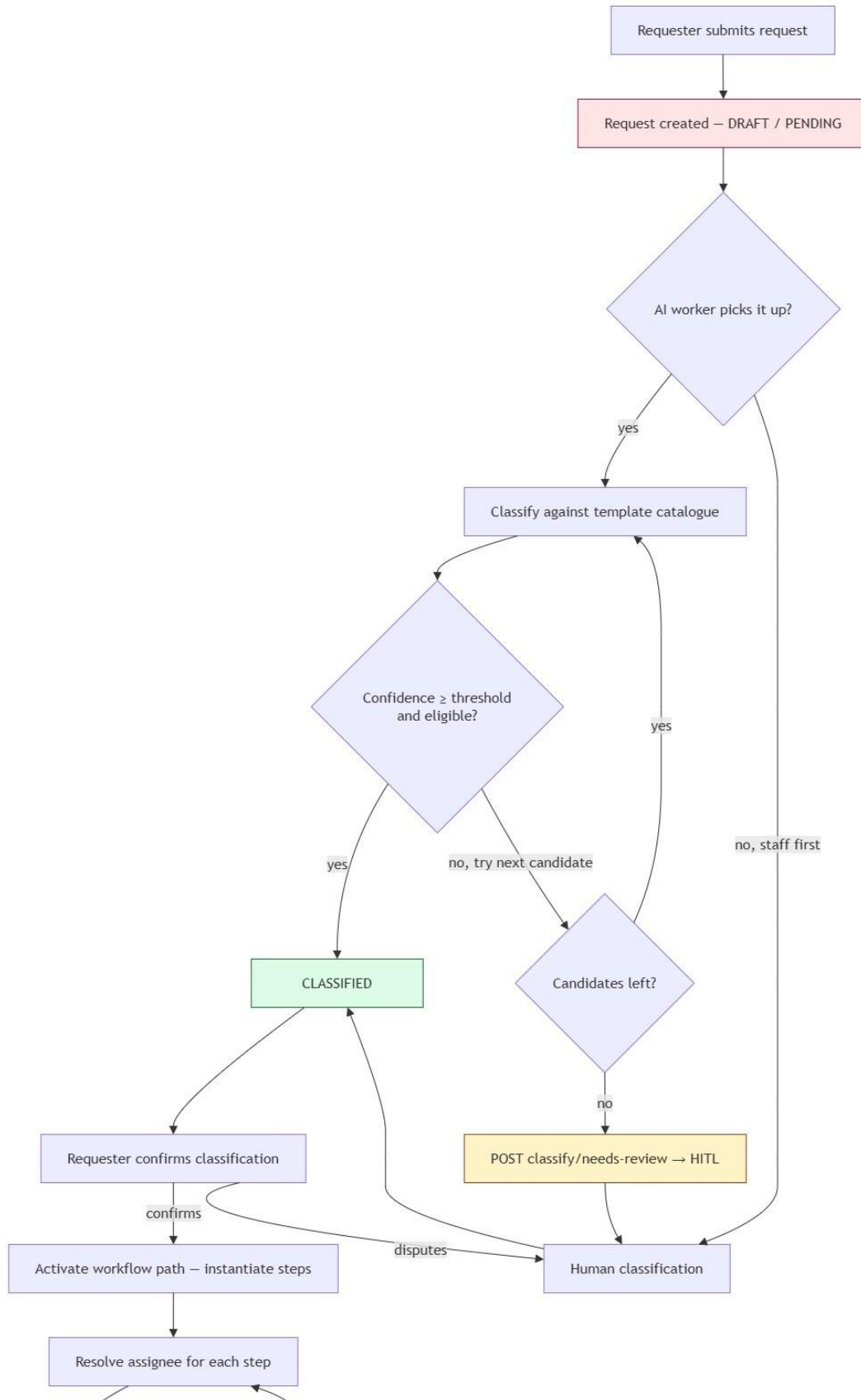
يصف هذا المخطط المعمارية ثنائية الفروع (dual-encoder) للمصنّف. الفرع الأول يعالج نص الطلب العربي، والفرع الثاني يعالج وثيقة وصفية واحدة لكل قالب (template) الفرعان يستخدمان أوزان MARBERTv2 نفسها (shared weights)، ثم masked mean pooling، ثم L2 normalisation، ثم cosine similarity، ثم softmax.

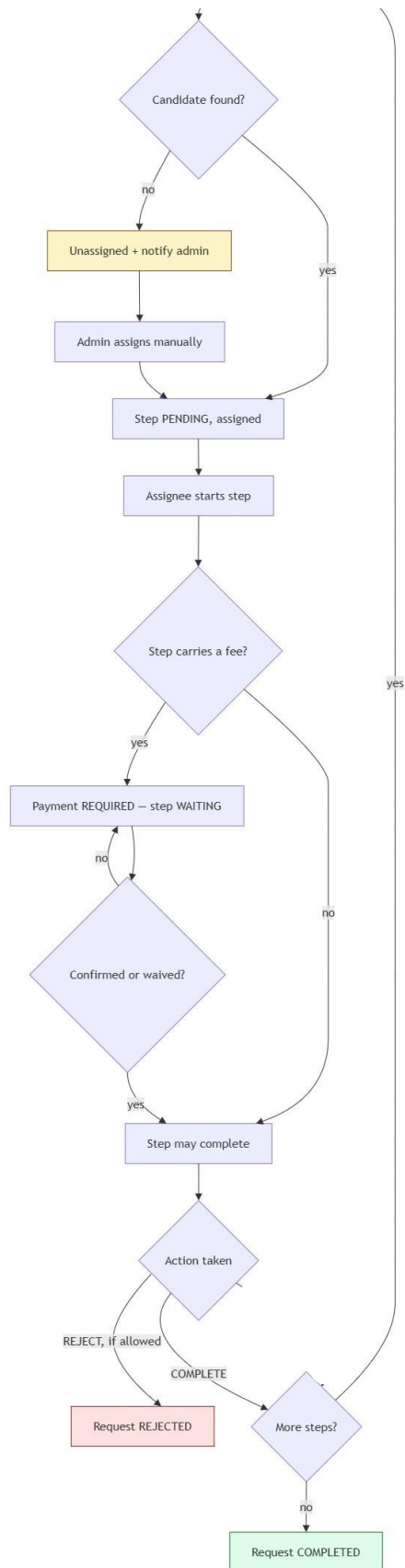
الفائدة المعمارية من هذا التصميم هي أنه يقبل إضافة أي template جديدة حيث أن ذلك يتطلب كتابة وثيقة وصفية له فقط (عند تعريف القالب الجديد من قبل الشخص المسؤول)، دون إعادة تدريب. لو استُخدم classifier تقليدي بطبقة softmax ثابتة، لكان كل قالب جديد يفرض إعادة تدريب كاملة، وهو شرط غير واقعي في نظام إداري تتغير متطلباته باستمرار.



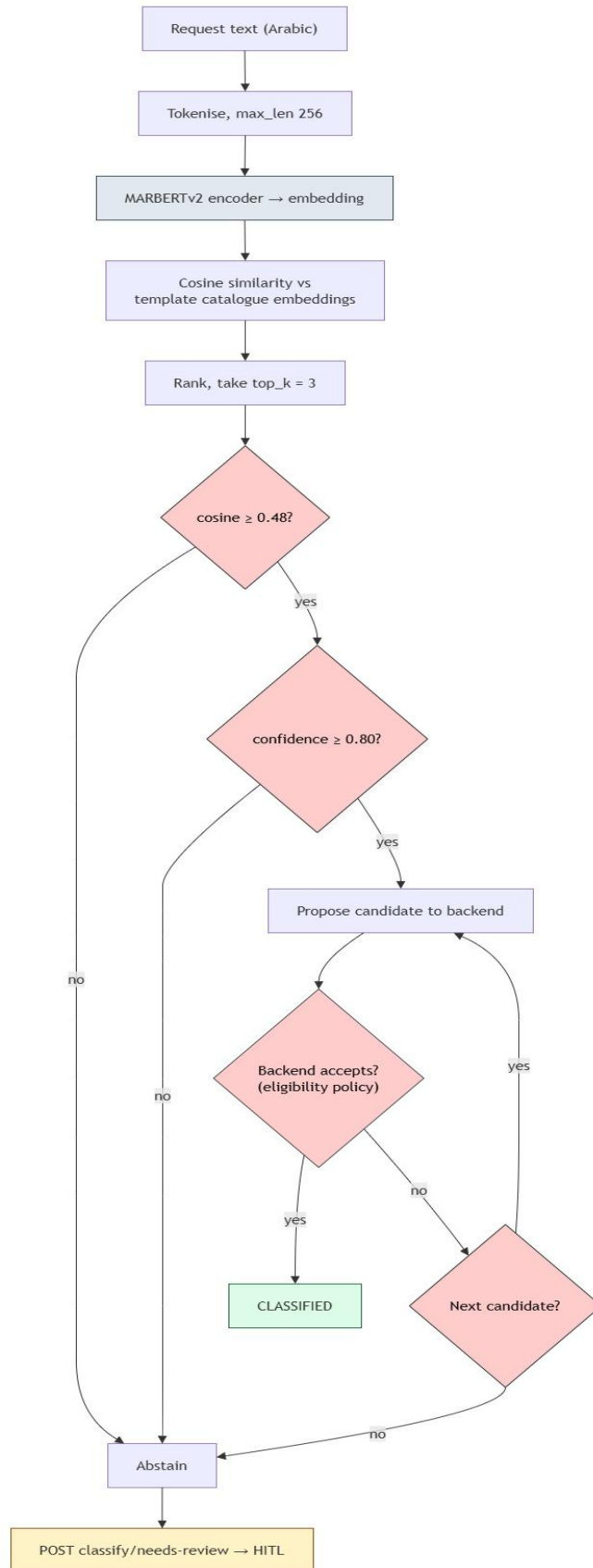
## Flow charts –5.2

- يوضح هذا المخطط دورة حياة الطلب من لحظة إنشائه حتى رفضه أو انقائه (قبوله).





- يوضح هذا المخطط آلية اتخاذ القرار في معالج المصنيف الذكي بالاعتماد على فلتين متكاملين:  
أولاً، يتم تطبيق حد التشابه المطلق (Cosine Floor عند 0.48) لفلتر النصوص العشوائية أو غير ذات الصلة (Out-of-Distribution)، ومنع خوارزمية الاحتمالات من إعطاء ثقة زائفة للنصوص المبهمة.  
ثانياً، يُطبق حد الثقة النسبي (Confidence Threshold عند 0.80) للتأكد من حسم النموذج لقراره بين القوالب المتشابهة، فإذا تجاوز الطلب الفلترين، يُقترح أفضل ثلاثة مرشحين (Top-k = 3) على الـ Backend بالترتيب.  
النظام الخلفي وحده هو مَنْ يملك صلاحية التحقق من الأهلية أي صلاحية مُقدم الطلب بتقديم هذا الطلب (Eligibility Policy)، فإذا رُفض المرشح الأول يتم اختبار الذي يليه. وتكمن قوة هذه الهيكلية في نهاية المسار: إذا انخفضت ثقة النموذج، أو استنفد المرشحون الثلاثة دون تطابق، لا يتوقف النظام برمجياً، بل يستدعي مسار المراجعة (POST /requests/:id/classify/needs-review)، لينتقل الطلب إلى طابور المراجعة البشرية ( HITL Queue). وبذلك يتحول القصور النموذجي إلى إجراء عمل مؤتمت، بدلاً من بقاء الطلب معلقاً.



## الفصل الثالث

# معمارية النظام المتبعة

يتضمن هذا الفصل شرح لمعمارية النظام المتبعة لكل جزء من أجزاء النظام.

### 1.3- ال Backend: معمارية طبقية مبنية على DDD

تم تقسيم ال backend إلى أربع طبقات رئيسية (Domain, Application, Infrastructure, Interface)، مع الالتزام بقاعدة اتجاه الاعتماديات يكون دائماً نحو الداخل فقط.

Domain Layer: تحتوي على قلب العمل وقواعده المنطقية التي لا تتأثر بتغير التقنيات. هذه الطبقة خالية تماماً من مكتبات مثل Prisma أو NestJS، مما يجعل اختبارها ممكناً دون الحاجة للاتصال بقاعدة بيانات.

Application Layer: تعمل كمنسق للعمليات ولا تتخذ قرارات العمل بنفسها. تُمثل كل حالة استخدام (Use Case) صنفاً (Class) مستقلاً، حيث تستدعي طبقة ال Domain لاتخاذ القرار، وتستخدم المنافذ (Ports) للتواصل مع العالم الخارجي.

Infrastructure Layer: تحتوي على ال Adapters التي تُنفذ المنافذ المطلوبة. هذه الطبقة هي المكان الوحيد الذي يتعامل مع التفاصيل التقنية الخارجية مثل أوامر قواعد البيانات (Prisma)، التواصل مع التخزين السحابي (MinIO)، أو استدعاء واجهات برمجة التطبيقات الخارجية (External APIs).

Interface Layer: تتكون من متحكمات التوجيه (Controllers) ومواصفات البيانات (DTOs) والحماية (Guards). تنتهي كل دالة فيها بتمرير الأمر إما إلى commandBus.execute أو queryBus.execute. خلو هذه الطبقة من قواعد العمل يجعل من إضافة واجهات جديدة مستقبلاً (مثل CLI أو GraphQL) أمراً سهلاً لا يتطلب إعادة كتابة أي منطق برمجي.

#### 1.1.3- استخدام CQRS في ال application layer

تم اتباع نمط CQRS في ال application layer لثلاثة أسباب عملية.

1. اختلاف الشكل بين القراءة والكتابة. الكتابة تحتاج aggregate كاملاً محملاً مع كل قواعده لتقول "نعم أو لا"، أما القراءة فتحتاج جدولاً بأعمدة من عدة كيانات. إجبار الاثنين على المرور بالنموذج نفسه يُنتج إما aggregates ضخمة تُحمّل بلا داع أو نموذج قراءة يُفسد domain .
2. صراحة الصلاحيات والaudit كل command هو فعل مثل ActOnStep، AssignStep، SettlePayment، ChangePriority، فيصبح ربط كل فعل بصلاحية ويسجل في EventLog مسألة مباشرة. لو كان كل شيء "update"، لأصبح التمييز بين من يملك حق التصنيف ومن يملك حق التنفيذ تمييزاً على مستوى الحقول لا على مستوى الفعل.
3. عمليات الاستعلام (Queries) تتجاوز كائنات المجال المعقدة (Domain Aggregates) عن قصد، وتتواصل مباشرة مع قاعدة البيانات عبر منافذ مخصصة (Query Ports). تُرجع هذه المنافذ هياكل بيانات مبسطة وجاهزة للعرض (View Types)، مما يُقلل من استهلاك الموارد ويُسرّع استجابة النظام. هذا التجاوز آمن تماماً ولا يكسر أي قاعدة منطقية (Invariants)، لأن عمليات القراءة لا تُعدّل حالة النظام.

## 2.1.3 - استخدام Ports and Adapters (Hexagonal)

تعتمد المعمارية على عزل النواة الداخلية للتطبيق (Application Core) عن العالم الخارجي. يتم تمثيل أي تبعية خارجية بـ "واجهة برمجية" (Interface/Port) تقع ضمن طبقة الـ Application، مثل ObjectStorage. في المقابل، تقع الـ Adapters التي تنفذ هذه الواجهات في طبقة البنية التحتية (Infrastructure)، مثل MinioObjectStorage. ويتم الربط بينهما عبر مُعرّفات حقن التبعية (Injection Tokens) مثل OBJECT\_STORAGE، مما يضمن أن الـ Handler في النواة لا يعرف أبداً اسم الكلاس التقني الذي ينفذ المهمة.

يحقق هذا النمط ثلاث فوائد هندسية جوهرية:

قابلية الاختبار المُطلقة (Testability): يمكن اختبار محرك التوجيه (Routing Engine) بالكامل عبر تمرير Adapter وهمي (Mock Adapter) لواجهة AssigneeDirectoryPort، مما يسمح باختبار الخوارزميات دون الحاجة لقاعدة بيانات حقيقية أو بيانات مبدئية (Seed Data).

المرونة أمام التغييرات الخارجية (Resilience): نظراً لأن الأنظمة الخارجية (كأنظمة الموارد البشرية) عُرضة للتغيير ولا نملك سيطرة عليها، فإن وجود Adapter وسيط يضمن أن أي تغيير في شكل الـ API الخارجي سيتطلب تعديل ملف واحد فقط، دون المساس بمنطق العمل الداخلي.

التوثيق الذاتي (Self-Documenting Contracts): يُمثّل الـ Port عقداً واضحاً يوثّق احتياجات النظام. فقراءة ملف assignee-directory.port.ts تكفي تماماً لفهم كل ما يطلبه محرك التوجيه من العالم الخارجي لأداء عمله.

### 3.1.3- الصلاحيات

يمرّ كل request بسلسلة واضحة:

1. JwtAuthGuard يتحقق من ال token ويستخرج هوية المستخدم.

2. PermissionsGuard يقرأ ال metadata ال decorator ويحسم القرار.

يوجد نوعان من ال decorators:

1. @RequirePermissions يعني AND، أي يجب توفر كل الصلاحيات المذكورة.

2. @RequireAnyPermission يعني OR، أي تكفي واحدة.

الصلاحيات نفسها لا تُلصق بالمستخدم مباشرة، بل تُحسب عبر السلسلة التالية:

User → UserRole → Role → RolePermission → Permission

وسجل UserRole يحمل departmentId و expiresAt، أي أن الصلاحية مقيّدة بوحدة ومحدّدة زمنياً، وهذا هو نفس الحقل الذي يعتمد عليه التوجيه.

يُضاف إلى ذلك حاجزان تشغيليان:

1. WorkingHoursGuard الذي يمنع أفعال الموظفين خارج أوقات العمل الرسمية.

2. قائمة STAFF\_IP\_ALLOWLIST التي تقيّد الوصول الإداري بعناوين معروفة.

### 4.1.3- الخدمات الخلفية (Background Services)

يوجد في النظام أربع آليات تعمل دون تدخل مستخدم، وكلها in-process providers داخل ال backend لا deployables منفصلة.

أولاً: SLA Monitor يعمل كل خمس عشرة دقيقة افتراضياً (SLA\_SWEEP\_MINUTES) بعد تأخير إقلاع مقداره خمس عشرة ثانية، ويحتسب المهل بساعات العمل الرسمية عبر BusinessHoursService لا بالساعات المطلقة، ثم يُصنّف كل خطوة إلى ON\_TRACK أو AT\_RISK أو BREACHED ويُصدر التنبيهات.

ثانياً: Storage Reconciliation يسمح bucket التخزين بحثاً عن ملفات لا يشير إليها أي سجل في قاعدة البيانات. يعمل كل أربع وعشرين ساعة، أي يترك مهلة أمان مقدارها أربع وعشرون ساعة قبل وصف أي ملف باليتيم، ولا يحذف شيئاً بل يُقرّر فقط. سبب هذا التحفظ أن غياب السجل هو نفس ما يبدو عليه الخطأ البرمجي أو الاستعادة من نسخة احتياطية، وثمن

حذف خاطئ هو فقدان مستند شخص حقيقي. ويمكن تشغيل المسح يدوياً عبر-POST /maintenance/storage-reconciliation بصلاحيه system.monitor.

ثالثاً: Notification Retention حذف الإشعارات الأقدم من ثلاثين يوماً (NOTIFICATION\_RETENTION\_DAYS) يقوم بمسح (scan) كل أربع وعشرين ساعة.

رابعاً: Dependency Health يفحص PostgreSQL و MinIO فقط، على التوازي، مع مهلة خمس ثوانٍ لكل منهما. الاختصار على هذين الاثنين لأنه إذا سقط أحدهما تسقط كل عملية ذات معنى.

### 5.1.3- الإشعارات (Notifications)

تعمل الإشعارات على مستويين. المستوى الأول هو الحفظ: كل إشعار هو سجل في جدول Notification ، فلا يفقده المستخدم إن كان غير متصل. والمستوى الثاني هو البث الفوري عبر Server-Sent Events.

اختير SSE لا WebSocket لأن الاتصال أحادي الاتجاه بطبيعته: الخادم يُبلغ والمتصفح يستمع فقط، و SSE يعطي إعادة الاتصال التلقائية مجاناً ويعمل فوق HTTP دون بروتوكول إضافي.

معمارياً، يتم تصدير الإشعارات عبر مكون NotificationEmitter في طبقة التطبيق، وتُثبت عبر منفذ NotificationStreamPort. ال Adapter الحالي يعتمد على الذاكرة (In-Memory)، مما يعني أن البث الفوري يعمل بكفاءة على خادم واحد، تبرز هنا أهمية المنافذ (Ports)؛ فعند الحاجة للتوسع باستخدام عدة خوادم، كل ما يتطلبه الأمر هو استبدال هذا المحوّل بآخر يعتمد على تقنية مشتركة مثل Redis ، دون المساس بسطر واحد من شيفرة التطبيق الأساسية.

### 6.1.3- التوجيه

يعتمد النظام على فصل قوالب العمل عن حالتها التنفيذية؛ فبينما يمثل WorkflowPath و WorkflowStep مخططات قابلة للتعديل، يمثل RequestStepInstance نسخة تنفيذية مُجمّدة غير قابلة للتغيير. ولضمان امتلاك كل قالب (Template) مسار عمل نشط وحيد، لم يتم الاكتفاء بمنطق التطبيق فقط، بل فُرض قيد على مستوى قاعدة البيانات ذاتها. كما تتم عملية استبدال مسار بآخر عبر معاملة موحدة (Database Transaction) لضمان إبطال القديم وتفعيل الجديد بآن واحد دون أي تضارب.

تم تصميم خوارزمية التوجيه لتكون لحظية (Lazy) ؛ فعند تأكيد الطلب، تُسند الخطوات الجاهزة للعمل فوراً فقط، وتُوجّه الخطوات اللاحقة تبعاً لحظة تحررها. يضمن هذا النهج احتساب أحمال العمل وصلاحيات التفويض (Delegation) بناءً على المعطيات الحية لحظة التنفيذ، متفادياً الاعتماد على بيانات قديمة قد تتغير (مثل تغيير المناصب خلال فترة سير الطلب).

علاوة على ذلك، فُرض تقييد صارم يَحصر الأفعال المخصصة للسُوح للخطوات بالأفعال النهائية (Terminal Actions) التي تُنهي حالة الخطوة فعلياً. يتم تطبيق هذا القيد بثلاث طبقات دفاعية: عند التعريف في واجهة المستخدم، وفي مُعالج الأوامر (Handler)، ولحظة التنفيذ الفعلي. يمنع هذا التقييد النظام من إغلاق خطوة بشكل خاطئ والانتقال للخطوة التالية في حال استخدام أفعال تُوحي باستمرار العمل. وتجدر الإشارة إلى أن هذا القيد هو إجراء مرحلي؛ حيث تم تصميم النظام بحيث يمكن توسيعه مستقبلاً لدعم الأفعال غير النهائية (Non-Terminal Actions) بمجرد بناء المنطق البرمجي الخاص بها لإدارة دورات العمل الفرعية دون إغلاق الخطوة الأساسية.

## 2.3- ال Frontend

تم بناء الواجهة الأمامية باستخدام إطار العمل **Next.js** ومعمارية ال **App Router**، ولضمان صيانة الكود وقابليته للتوسع، قُسمت المسؤوليات برمجياً إلى ثلاث طبقات واضحة:

1. **طبقة ال API**: تحتوي حصراً على دوال التواصل عبر بروتوكول HTTP باستخدام (Axios) والتعامل مع المسارات، دون أي منطق يتعلق بواجهة المستخدم.

2. **طبقة ال Hooks**: مبنية بالاعتماد على مكتبة (TanStack Query (React Query)، وتتولى إدارة جلب البيانات، التخزين المؤقت (Caching) وإبطال التخزين (Cache Invalidation) تلقائياً عند حدوث تغييرات (Mutations) في النظام.

3. **طبقة الواجهات والمكونات (Pages & Components)**: تتكون من مكونات مرئية معزولة تماماً عن تفاصيل الشبكة أو ال HTTP، حيث تعتمد فقط على ال Hooks للحصول على البيانات وحالات التحميل (Loading/Error States). يضمن هذا الفصل المعماري الكفاءة العالية ويسهل عملية الاختبار وتطوير الواجهات.

## 3.3- خدمة تصنيف الطلبات (AI Model)

فُصلت خدمة التصنيف عن ال backend في deployable مستقل بلغة Python لثلاثة أسباب: اختلاف بيئة التشغيل (torch) و (transformers)، واختلاف ملف الأداء (تحميل النموذج ثقيل والاستنتاج مكلف)، والأهم أن سقوط النموذج يجب أن لا يُسقط النظام الإداري بأكمله.

التكامل عن طريق pull لا push: ال worker يستقضي الطلبات غير المصنّفة كل ثلاثين ثانية، ويُحدّث نسخته من ال template catalogue كل عشر دقائق مع الاحتفاظ بال UUIDs الحقيقية للtemplates ، ثم يرسل النتيجة إلى ال backend. هذا الاتجاه الواحد للاستدعاء يعني أن ال backend لا يعرف عنوان خدمة ال AI ولا ينتظرها، فتوقّف الخدمة يعني تراكمًا في طابور التصنيف البشري لا فشلاً في إرسال الطلبات.

القرار الأهم على مستوى النموذج هو الامتناع (abstention) التصنيف الخاطئ في نظام إداري لا يُنتج "نتيجة أقل جودة" بل يُرسل الطلب إلى الموظف الخطأ ويُخفي الخطأ داخل مسار يبدو سليماً. لذلك يمتنع النموذج عند انخفاض التشابه أو الثقة، ويُسلّم الطلب إلى طابور ال HITL.

### 3.3- تفصيل أجزاء النظام وطريقة تنفيذ التكامل فيما بينها

#### 1.3.3- تكامل الواجهة الأمامية مع الواجهة الخلفية

يُمثّل ال REST فوق بروتوكول HTTP العقد الأساسي للتواصل بين ال Frontend وال Backend، وتتم المصادقة عبر تمرير JWT Token في ترويسة Authorization لضمان سلامة المدخلات، تتحقق كائنات نقل البيانات (DTOs) من صحة الحمولات الواردة باستخدام ValidationPipe مع تفعيل خيار whitelist، مما يضمن إسقاط أي حقل غير مُعرّف تلقائياً بدلاً من تمريره إلى طبقات النظام الداخلية.

أما على صعيد معالجة الأخطاء، فقد تم توحيد الاستجابات عبر DomainExceptionHandler، والذي يترجم استثناءات المجال والتطبيق إلى رموز HTTP معيارية وثابتة (مثل: EntityNotFoundError إلى 404، ForbiddenActionError إلى 403، ConcurrentModificationError إلى 409، و UpstreamUnavailableError إلى 502). أما الأخطاء غير المتوقعة، فتُعاد كاستجابة موحدة تحتوي حصراً على مُعرّف تتبع (traceId)، بينما يُسجّل الاستثناء بكامل تفاصيله في سجلات الخادم (Logs). يضمن هذا العزل ألا تظهر تفاصيل الشيفرة (Stack Traces) أو أخطاء قاعدة البيانات للمستخدم النهائي، مع منح المهندس مرجعاً دقيقاً يربط ما رآه المستخدم بسجل الخادم، وهو أسلوب أثبت فاعليته العالية في تشخيص أعطال هذا المشروع.

#### 2.3.3- تكامل الإشعارات الفورية (SSE)

لتقديم تجربة إشعارات حية، يفتح المتصفح اتصال EventSource مستمر مع الخادم. ونظراً لأن واجهة EventSource القياسية لا تدعم إضافة ترويسات مصادقة مخصصة، تتم المصادقة عبر تذكرة قصيرة الأجل (Short-lived Ticket) تُصدر حصراً لهذه الغاية من StreamTicketService.

عند حدوث حدث يستوجب الإشعار، يُنشر الإشعار عبر القناة المتصلة ويُحفظ في الوقت ذاته في قاعدة البيانات. بمجرد استلام الواجهة للإشعار، تقوم فوراً بإبطال التخزين المؤقت المرتبط به (Cache Invalidation)، مما يؤدي إلى تحديث الأرقام وقوائم المهام في واجهة المستخدم تلقائياً، دون الحاجة للجوء إلى تقنية الاستعلام المستمر (Polling).

### 3.3.3- تكامل خدمة الذكاء الاصطناعي

تعتمد الخدمة في تواصلها مع النظام على ثلاث نقاط تكامل رئيسية:

1. تسجيل الدخول باستخدام حساب خدمة مخصص (AI\_SERVICE\_EMAIL) للحصول على JWT.

Token

2. جلب فهرس القوالب النشطة (Template Catalogue) بشكل دوري لتحديث نموذجها المحلي.

3. إعادة نتائج التصنيف إلى النظام الخلفي عبر المسار POST /requests/:id/classify/model، حيث يقوم

النظام حينها بتطبيق سياسات الأهلية على المرشح المقترح، أو إرسال الطلب عبر POST /requests/:id/classify/needs-review في حال امتناع النموذج أو استفاده للمرشحين.

النظام الخلفي لا يبادر بأي استدعاء نحو خدمة الذكاء الاصطناعي (Pull Architecture)، مما يضمن استقلالية الخدمات.

### 4.3.3- تكامل التخزين

لضمان الأداء العالي، لا تمر الملفات عبر الـ Backend أثناء عملية التنزيل. بدلاً من ذلك، يتحقق النظام من صلاحيات المستخدم، ثم يُصدر رابطاً مؤقتاً موقّعاً (Presigned URL) صالحاً لمدة زمنية قصيرة، ويتصل المتصفح بعدها بخوادم التخزين (MinIO) مباشرة. تُجنّب هذه الآلية واجهات الـ API حمل معالجة الملفات الكبيرة، مع الحفاظ على مركزية وأمان قرارات التفويض في مكان واحد.

كما تم تصميم تسلسل عملية الرفع كالتالي: تُكتب البيانات أولاً على خوادم MinIO، ثم يُحفظ السجل المرجعي في قاعدة البيانات. والسبب وراء هذه المقايضة (Trade-off) هو أن الاختيار بين الخطوتين سيُخلّف ملفاً يتيماً لا يشير إليه أحد (بمجرد مساحة مهدورة)، وهو ضرر أخف بكثير من الحالة المعاكسة التي تُنتج سجلاً يشير إلى ملف مفقود. وتعمل خدمة التسوية (Storage Reconciliation) كجزء مكمل لهذه المقايضة لتمشيط وحذف تلك الملفات اليتيمة.

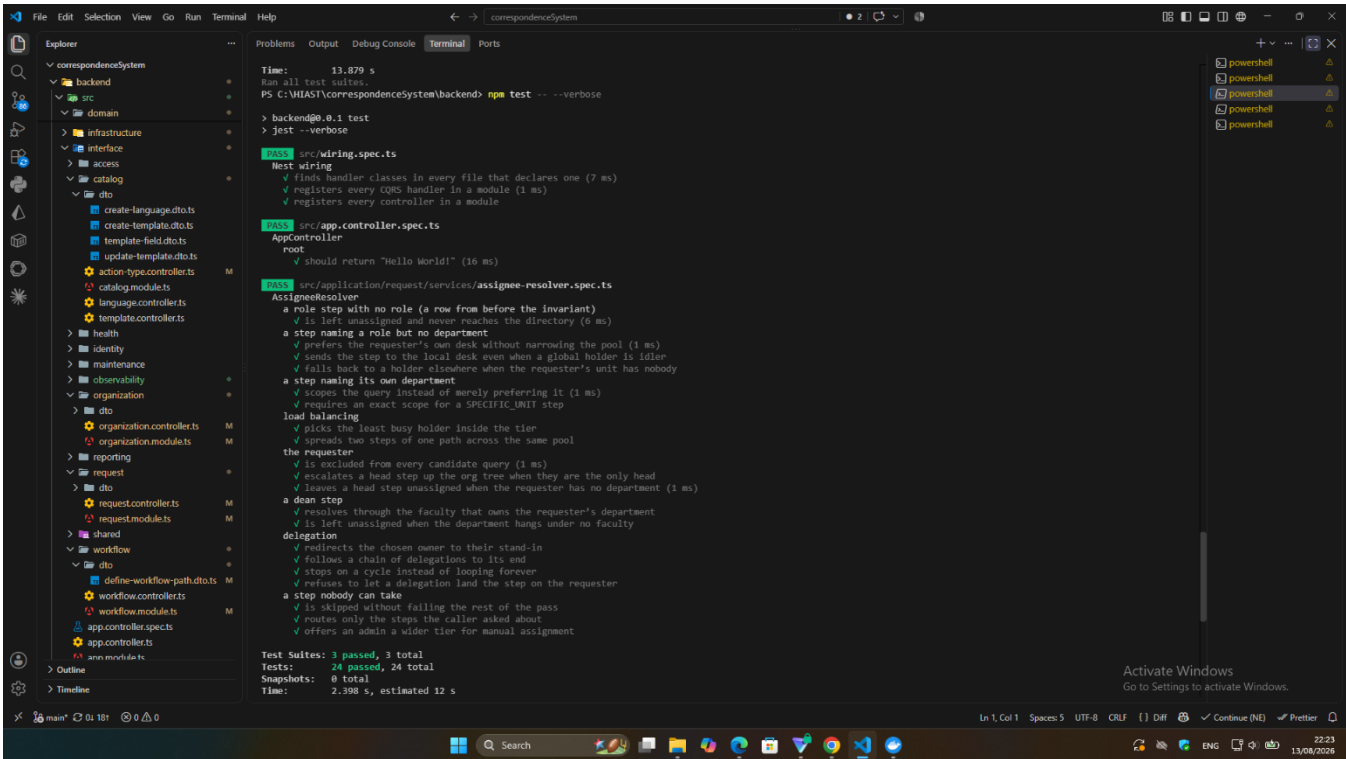
### 3.3.3- تطبيق الـ Transactions

لضمان اتساق البيانات المطلق، يتم تنفيذ كل أمر (Command) بمس أكثر من جدول داخل TRANSACTION\_RUNNER موحد لضمان خصائص الـ ACID. تتجلى أهمية ذلك في حالتين واقعتين من هذا المشروع:

- الأولى (استبدال المسار النشط): حيث يجب أن تتم عمليات إبطال المسار القديم، وتفعيل الجديد، وإلغاء ارتباطات الخطوات السابقة ككتلة واحدة (Atomic) وقد بينت التجارب السابقة أن فصلها كان يؤدي إلى انتهاك قيد الفردة (Unique Constraint) على `template_id`.
- الثانية (تنفيذ فعل على خطوة): تشمل تحديث حالة الخطوة، تسجيل الفعل (`RequestAction`)، توجيه الخطوة التالية، إصدار الإشعارات، وكتابة سجل التدقيق (`EventLog`) إن تنفيذ جميع هذه الخطوات في ذرة واحدة يضمن ألا يُحفظ التحديث دون إشعار ودون توثيق تاريخي، لأن تسجيل إشعار متأخر خطوة لم تُحفظ أشد خطراً من غياب الإشعار كلياً.

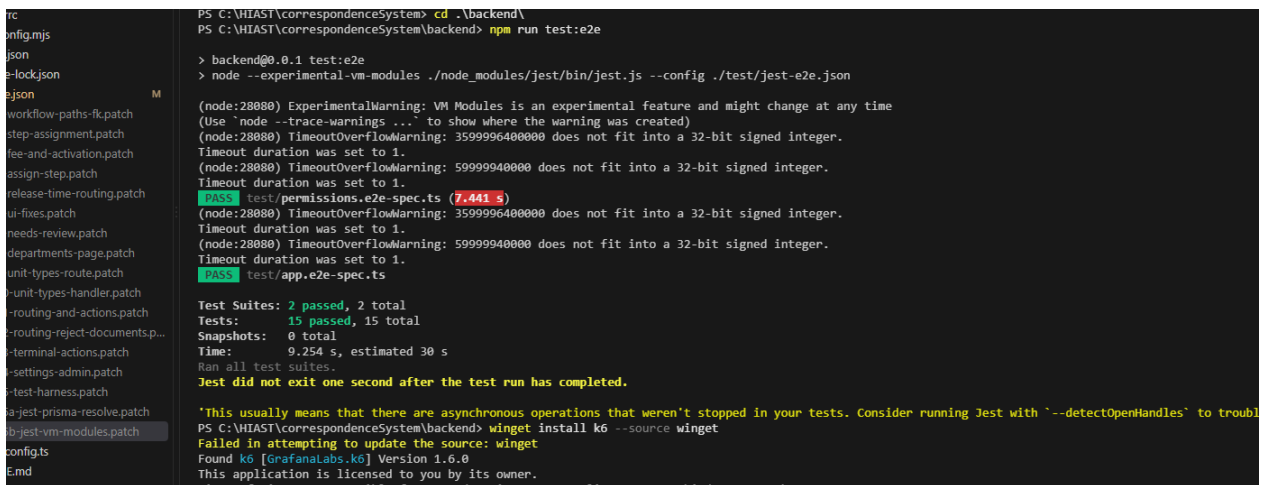
### 3.3- الاختبارات

- ✓ (unit tests) نُفذت اختبارات الوحدة على محرك إسناد الخطوات `AssigneeResolver`، وهو أكثر أجزاء النظام تعقيداً في منطقته. تغطي الاختبارات ثماني حالات: الخطوة القديمة الناقصة الدور، وتفضيل المكتب المحلي على العام، والرجوع إلى حامل الدور العام عند غياب المحلي، وتقييد النطاق للخطوات الموجهة إلى وحدة بعينها، وموازنة الحمل بين حاملي الدور، واستبعاد مُقدّم الطلب من المرشحين، وتصعيد خطوة رئيس القسم وعميد الكلية عبر الشجرة التنظيمية، وسلوك التفويض بما فيه كشف الحلقات المغلقة.
- يُنَفَّذ معها اختبار بنيوي لربط الوحدات `wiring.spec.ts`، يفحص الشيفرة المصدرية نصياً ويتحقق من تسجيل كل مُعالج أوامر واستعلامات وكل مُتحرّك في وحدة `Module` تابعة له. وقد أُضيف هذا الاختبار بعد خطأ فعلي واجهناه في أثناء التطوير: مُعالج مُرَبَّن بـ `@QueryHandler` لكنه غير مسجَّل في وحدته، فُيترجم البرنامج وينطلق دون أي إنذار، ثم يفشل عند أول طلب حقيقي بالرسالة `No handler found for the query`. يكشف الاختبار البنيوي هذه الفئة من الأخطاء كلها قبل التشغيل.
- يوضح الشكل نجاح جميع الاختبارات:



✓ (integration tests) تُنشئ هذه الاختبارات نسخة كاملة من التطبيق داخل بيئة الاختبار، مع الوسائط والخوِّاس ومُرشِّح الاستثناءات ذاتها المستخدمة في الإنتاج، وتُخاطبها عبر بروتوكول HTTP الحقيقي. وتعمل مقابل قاعدة بيانات منفصلة `correspondence_test`، مع دالة تفرغ `truncateAll` تعيد الحالة إلى نقطة معلومة قبل كل اختبار. يتحقق اختبار الإقلاع من أن الحاوية تُبنى بنجاح، وأن `/health` يستجيب، وأن المسارات المحمية ترفض الزائر المجهول. وأهمية هذا الاختبار أنه يفشل لأسباب مختلفة تماماً عن أسباب فشل اختبارات المسارات، فهو يكشف نقص التبعيات وتعدُّر الوصول إلى قاعدة البيانات.

يوضح الشكل نجاح جميع الاختبارات:



✓ (security tests) صيغت اختبارات الأمن على هيئة مصفوفة صلاحيات تسأل السؤال السالب: هل يُرفض المستخدم غير المصرّح له، وبالرمز الصحيح؟ والتميز بين الرمزين جوهري: الرمز 401 يعني أن هوية المستخدم غير معروفة، والرمز 403 يعني أن هويته معروفة وصلاحيته غير كافية. أما ظهور الرمز 200 في موضع يُنتظر فيه 403 فهو ثغرة تصريح فعلية.

تغطي المصفوفة ثلاثة أدوار: الزائر المجهول، والمراجع، والمدير. وتتحقق من رفض الرمز المزور، ومن إجابة 404 لمفتاح إعدادات غير مدعوم، ومن إجابة 400 لسياسة ساعات عمل غير صحيحة ينتهي وقتها قبل بدايتها. واقتصر في حالات المدير على الصلاحيات غير المقيّدة زمنياً، لأن حارس ساعات العمل WorkingHoursGuard يَحصر صلاحيات أخرى نافذة زمنية، فلو اختُبرت لفشل الاختبار بسبب الساعة لا بسبب الشيفرة.

يوضح الشكل نجاح جميع الاختبارات:

```
Problems  Output  Debug Console  Terminal  Ports
PS C:\HIAST\correspondenceSystem\backend> powershell -ExecutionPolicy Bypass -File test/security/manual-checks.ps1

Security checks against http://localhost:3000

(signed in as administrator and reviewer)

1. Authentication -- the door is shut by default
PASS anonymous may read the public liveness probe HTTP 200
PASS anonymous is refused a protected route HTTP 401
PASS anonymous is refused the permission list HTTP 401
PASS a token with a tampered signature is refused HTTP 401
PASS a malformed token is refused, not crashed on HTTP 401

2. Authorisation -- a known user with insufficient rights
PASS reviewer is refused a user.manage route HTTP 403
PASS reviewer is refused a system.monitor route HTTP 403
PASS reviewer is refused detailed health HTTP 403
PASS administrator may call the same route HTTP 200

3. Injection -- untrusted input reaches the database as data
PASS a SQL payload in a filter is inert HTTP 404
PASS a SQL payload in a path parameter is rejected cleanly HTTP 404

4. Input validation -- the write path refuses nonsense
PASS a working day that ends before it starts is refused HTTP 400
PASS an unknown settings key answers 404, not 500 HTTP 404

5. Horizontal privilege -- one user reading another user's data
SKIP pass -RequestId <id of a request owned by someone else>

6. Rate limiting -- brute force is throttled
eight rapid wrong passwords -> 401 401 401 429 429 429 429
PASS the login route throttles repeated attempts HTTP 429

14 passed, 0 failed
```

لم يتم تنفيذ load tests بعد.

### 3.3- بعض واجهات النظام

#### 1. واجهة تسجيل الدخول:

ICS

**Welcome again**

Sign in to continue to your requests.

Email

user@gmail.com

Password

Sign In

Activate Windows  
Go to Settings to activate Windows.

#### 2. واجهة إنشاء حساب:

ICS

**Welcome to ICS**

Create an account to submit and track your requests.

Full name (Arabic)

Full name (English)

Email

user@gmail.com

Phone

Password

Purpose

— select —

STUDENT\_ADMISSION

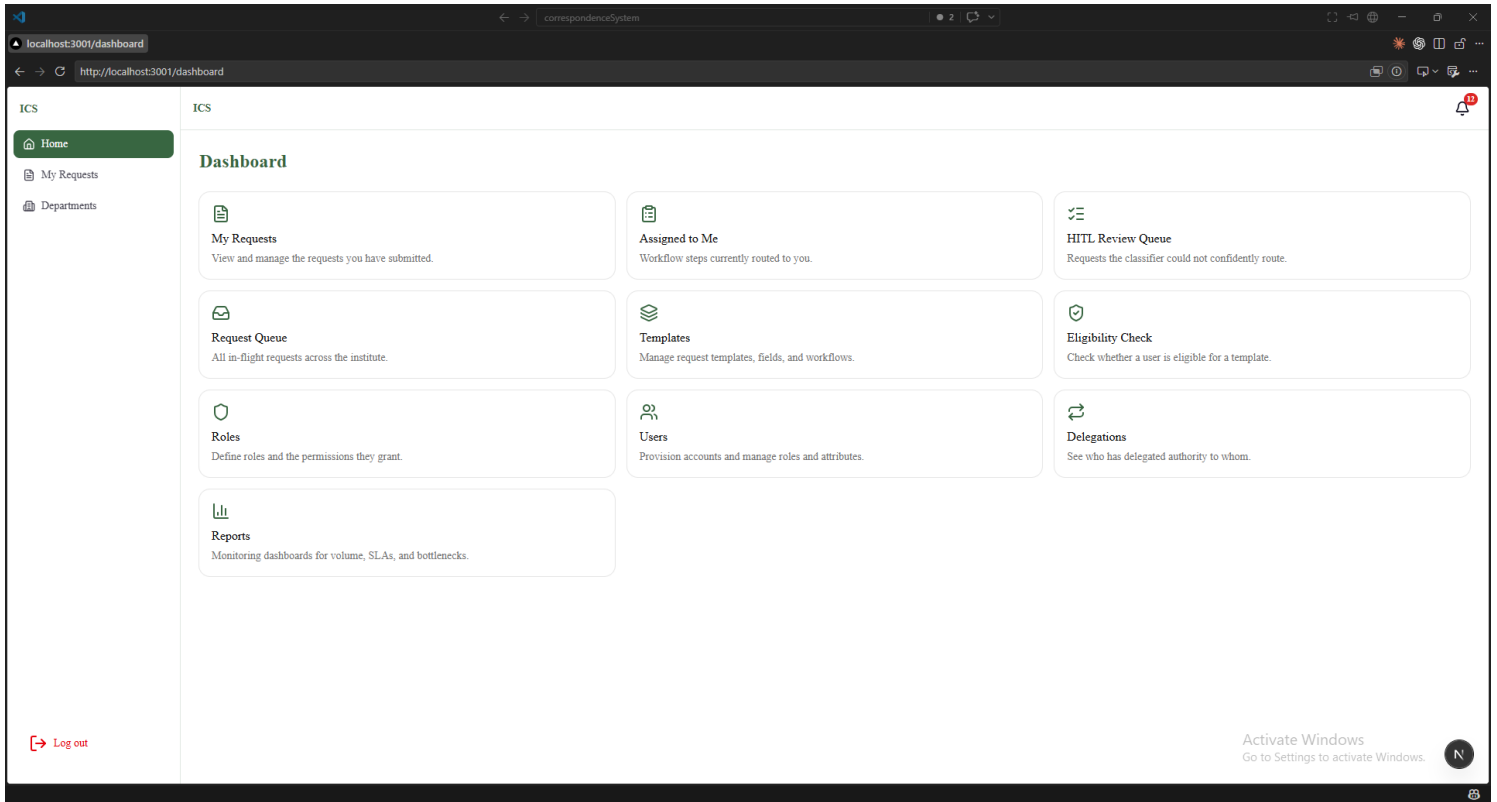
GRADUATE\_PROGRAM

JOB

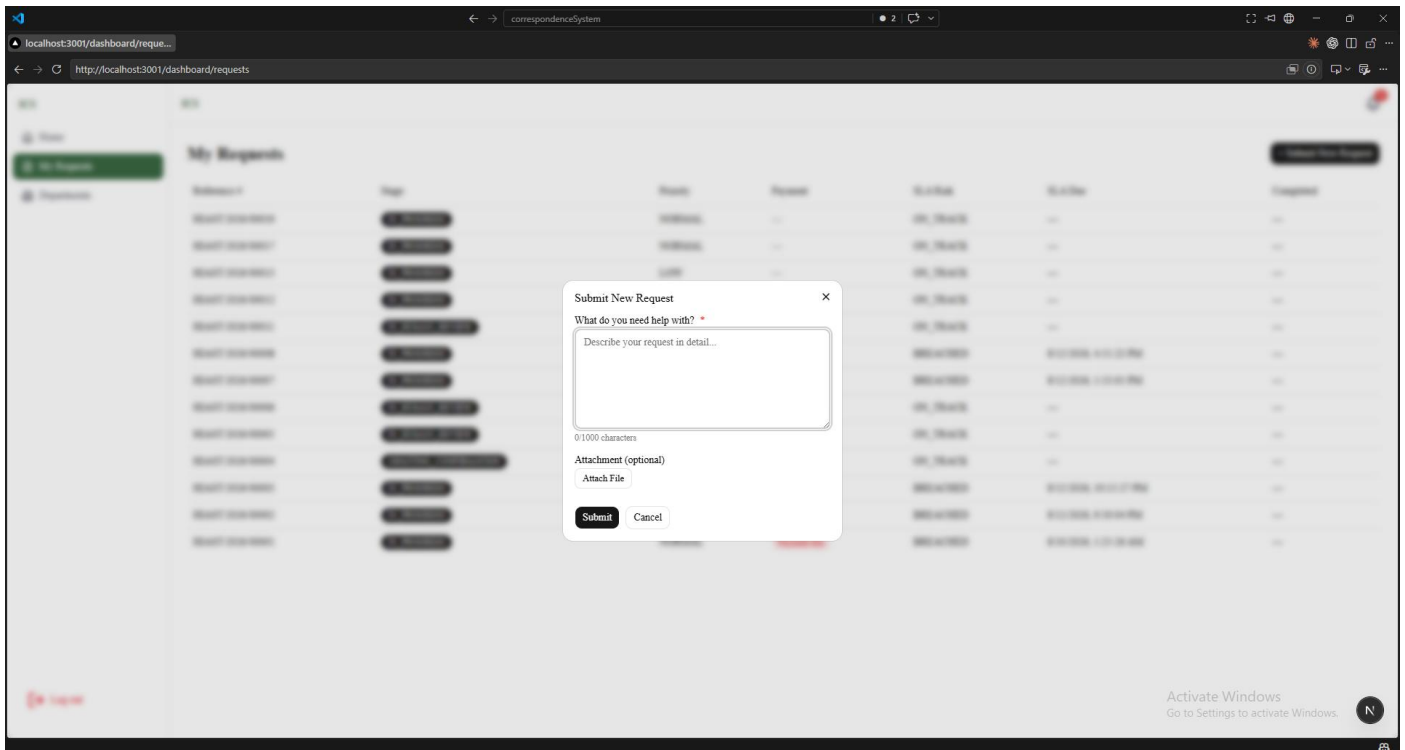
Sign Up

Activate Windows  
Go to Settings to activate Windows.

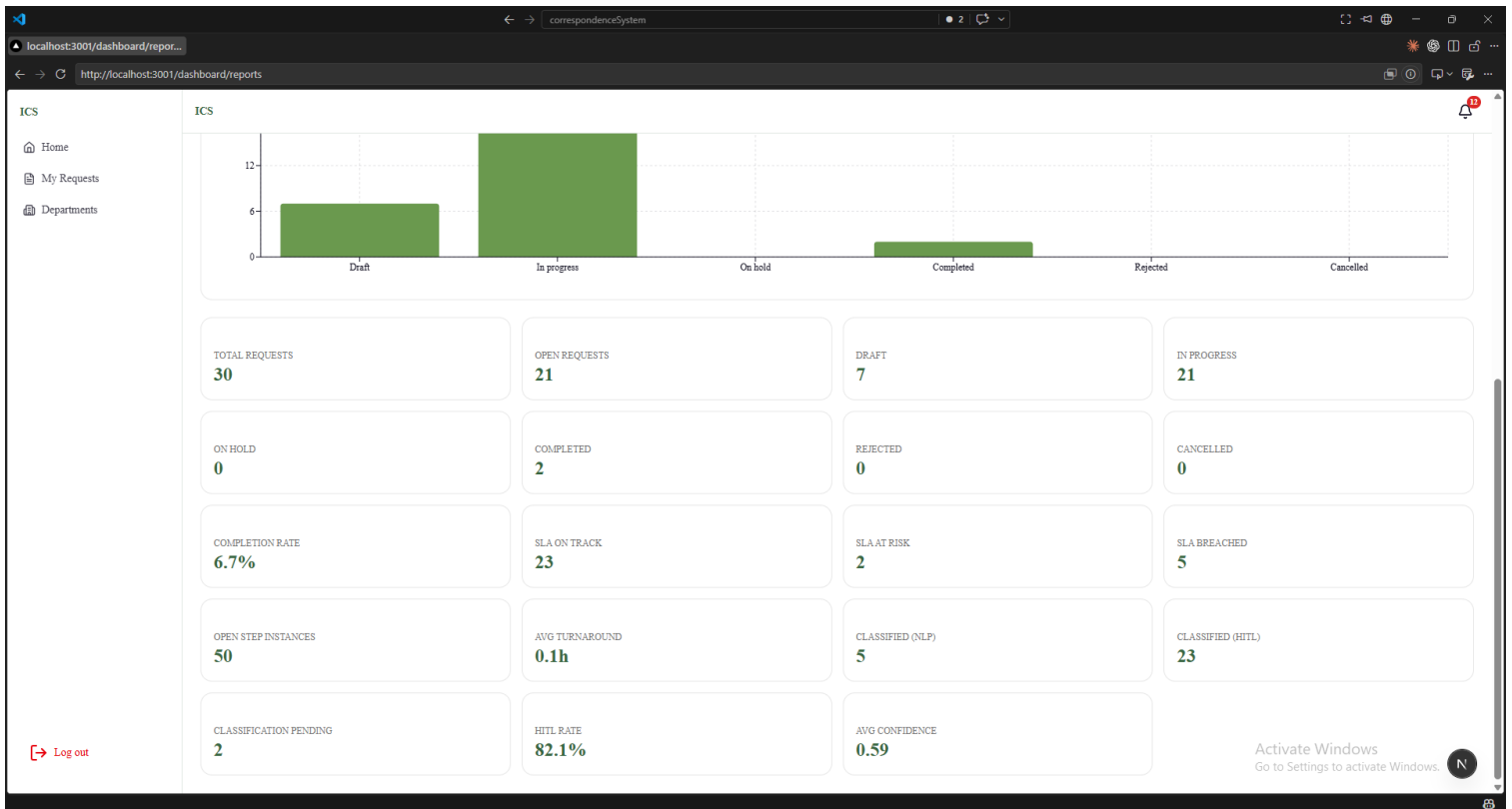
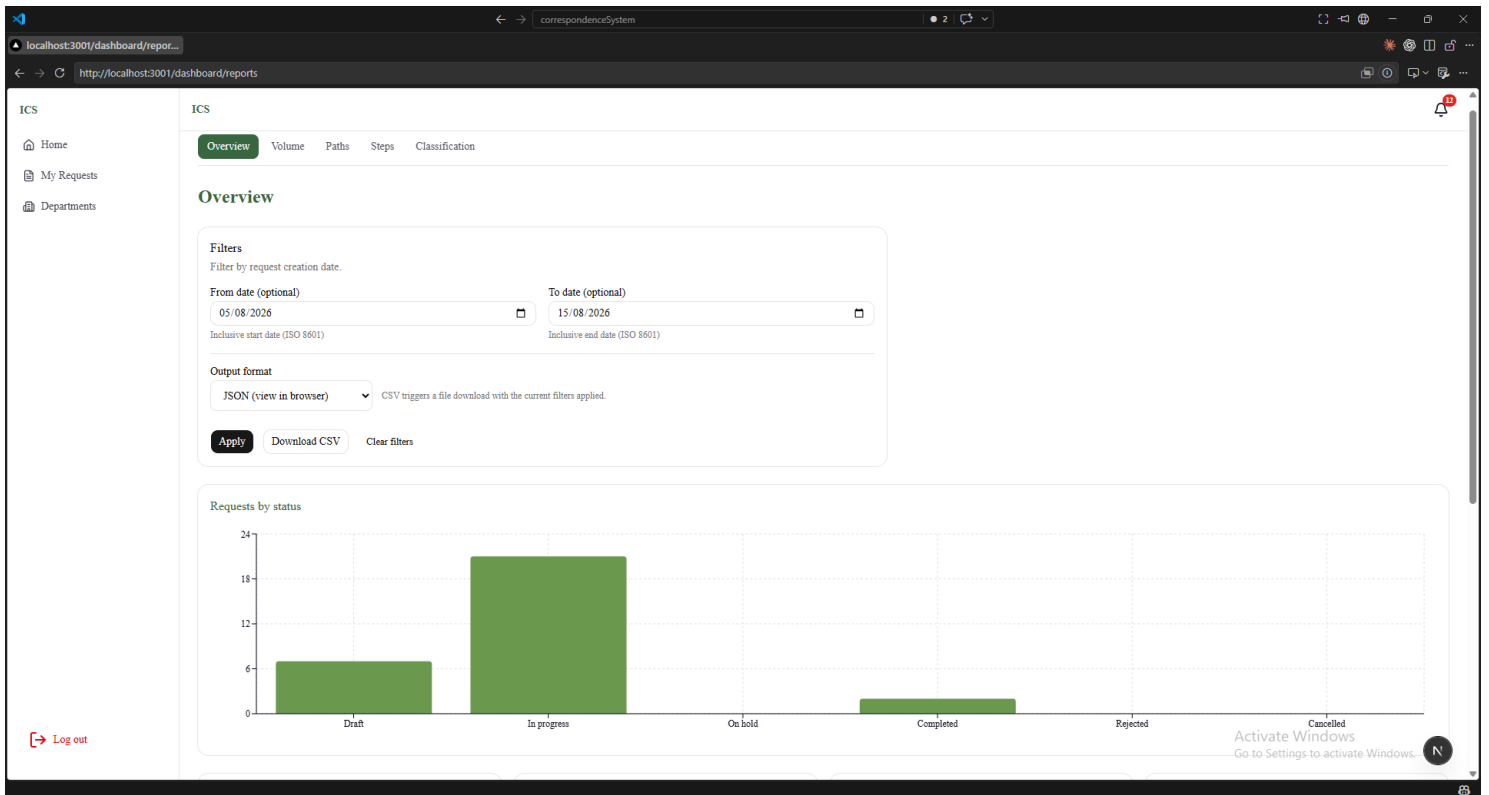
### 3. الواجهة الرئيسية (Dashboard):



### 4. واجهة تقديم الطلبات:



## 5. واجهة تقارير الأداء:



## 6. واجهة تأكيد الطلب (بعد تصنيفه):

The screenshot shows a web browser window with the URL `http://localhost:3001/dashboard/requests/019ffd0a-f748-71bc-9fc8-a8470ffb6707/edit`. The page is titled 'ICS' and has a sidebar with 'Home', 'My Requests', and 'Departments'. The main content area displays a form for 'طلب نقل اختصاص' (Request for Transfer of Authority). The form includes a warning message: 'Review the values below and confirm, or request reclassification if this is the wrong template.' Below this, there are three input fields: 'الاختصاص الحالي' (Current Authority), 'الاختصاص المطلوب' (Required Authority), and 'السبب' (Reason). A message below the fields states: '1 required field still need attention.' At the bottom of the form are three buttons: 'Cancel', 'Request Reclassification', and 'Confirm'. The 'Confirm' button is highlighted in black. In the bottom right corner, there is a 'Log out' link and an 'Activate Windows' watermark.

The screenshot shows the same web browser window as the previous one, but the main content area now displays a message: 'This request can't be edited right now.' Below this message is a button labeled 'Back to My Requests'. The sidebar and 'Log out' link remain the same. In the bottom right corner, there is a 'Log out' link and a 'Your request is now in progress' message, which is a notification from the system.

## 7. واجهة تعريف مسار عمل جديد لقلب معين:

localhost:3001/dashboard/templ... correspondenceSystem

http://localhost:3001/dashboard/templates/00000000-0000-7000-8000-000a00000002/workflow-paths/new

ICS

Home  
My Requests  
Departments

ICS

**Define Workflow Path**  
Create a new workflow path for a template. Define the steps, their assignees, dependencies, and SLA settings. The path can be activated immediately or later.

Template \*  
طلب إدارية (Administrative leave request)

Name (Arabic) \*  
Name (English)  
Description (Arabic)  
Description (English)

Steps (1) [+ Add Step](#)

Step 1  
Key \*  
e.g., review, approve, sign  
Unique identifier for this step (used in dependsOn). Lowercase, no spaces.  
Name (Arabic) \*  
Name (English)  
Description (Arabic)  
Description (English)

[Log out](#)

Activate Windows  
Go to Settings to activate Windows.

localhost:3001/dashboard/templ... correspondenceSystem

http://localhost:3001/dashboard/templates/00000000-0000-7000-8000-000a00000002/workflow-paths/new

ICS

Home  
My Requests  
Departments

ICS

Assignee type \*  
Specific Unit (SPECIFIC\_UNIT)

Department \*  
— select department —

Default action type (optional, terminal only)  
— select action type —

SLA hours (optional)  
e.g., 24

☐ Pauses SLA  
Time spent in this step doesn't count toward SLA.

Allowed action types (optional)  
موافقة (Approve) [APPROVE]  
رفض (Reject) [REJECT]  
Hold Ctrl/Cmd to select multiple. Terminal action types only (actions that end a request's journey).

Fee amount (optional)  
e.g., 5000  
Charged when this step is started. The step cannot be completed until the fee is confirmed or waived. Leave empty for a free step.

Currency  
SYP  
3-letter ISO code. Defaults to SYP.

Depends on steps (optional)  
Hold Ctrl/Cmd to select multiple. Only steps with keys appear here.

☐ Activate immediately (make this the default path for the template)  
Only one active path per template. Activating will deactivate any existing active path.

[Create Workflow Path](#) [Cancel](#)

[Log out](#)

Activate Windows  
Go to Settings to activate Windows.